Client: Android Studio (CHPlay)

Backend: NodeJS, Redis
DB: Redis, MongoDB

Service: user service, **chat service, media service**, audio call service, video call service, message service, search service, utility service

Cross-cutting service: Auth service, notification, monitor&logging

Api: Nginx reserve proxy (HTTPS)

Client: Android (Kotlin, Compose, Room, FCM, WebRTC), phát hành CH Play

Gateway: Nginx (:80 dev, :443 prod)

Service Node: user, chat, message, media, audio call, video call, search, utility

Cross-cutting: auth, notification, monitor & logging

Hạ tầng: MongoDB, Redis (kèm Streams), MinIO, coturn

Với danh sách này, mình khuyên làm theo thứ tự: Gateway → Auth + Identity → Chat + Message → Notification → Media → Call → Search → Utility (backup). Nếu thiếu thời gian thì cắt từ cuối lên.

Nơi lưu Vai tròDữ liệu

Room (SQLite, trên máy) Cache hiển thị + hàng đợi gửiTin nhắn, hội thoại, outbox, cursor sync

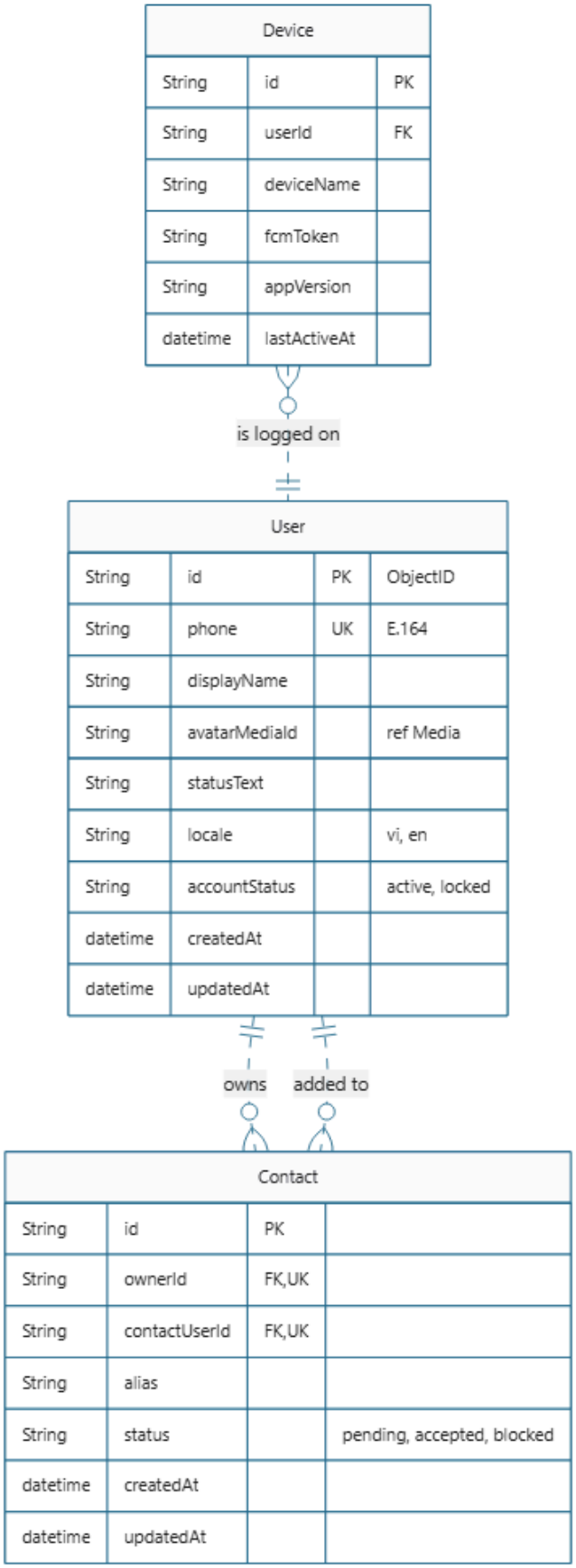
MongoDB Lưu bền, nguồn sự thậtMỗi service một database riêng (xem bên dưới)

Redis Dữ liệu tạm, có TTLOTP, session, presence, typing, rate limit, Streams (event)

MinIO File thậtẢnh, video, file backup

Room (máy) ⇄ Retrofit / WebSocket ⇄ Nginx ⇄ Message service ⇄ MongoDB

IdentityDB

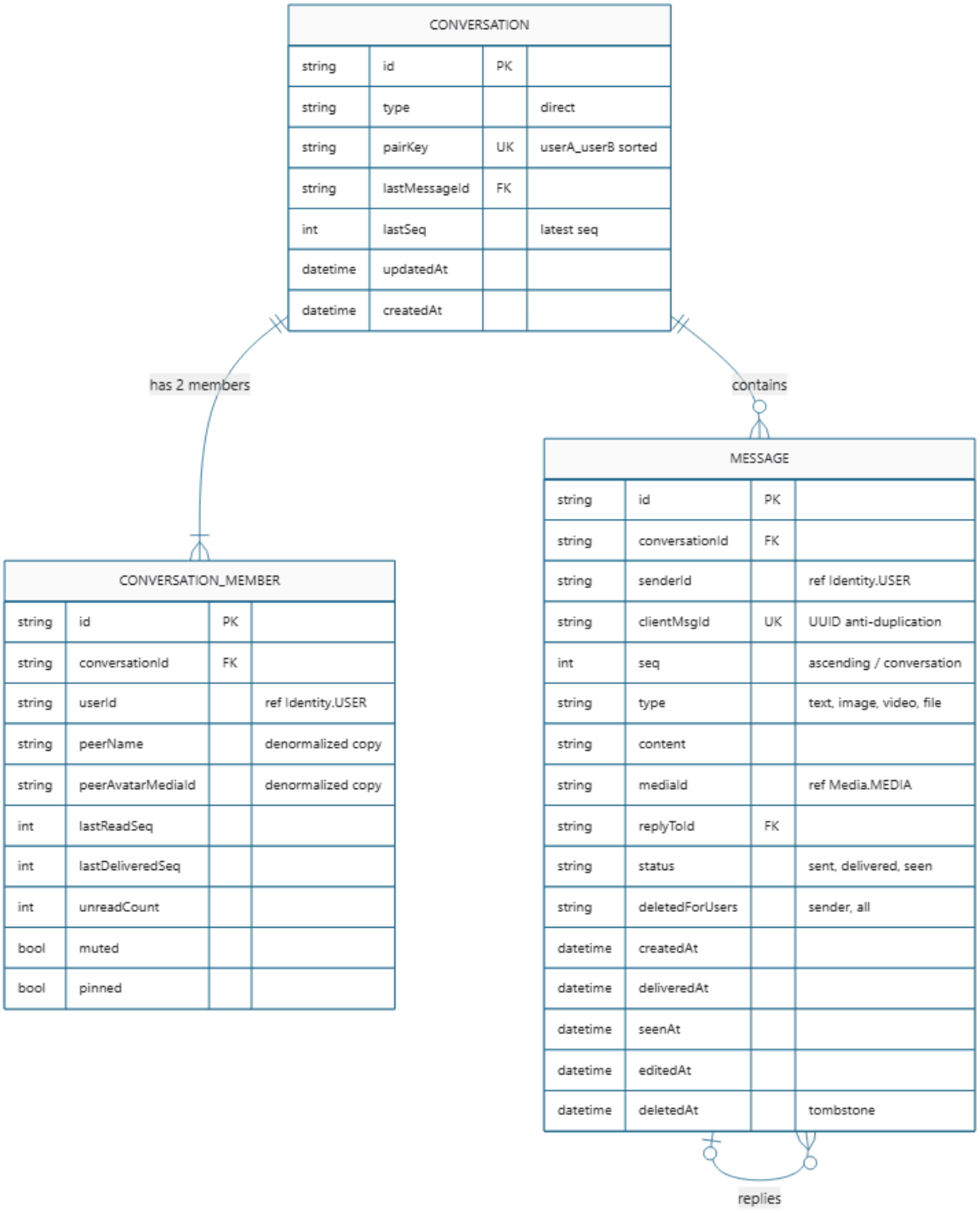

Device
String id PK
String userId FK
String deviceName
String fcmToken
String appVersion
datetime lastActiveAt
is logged on
User
String id PK ObjectID
String phone UK E.164
String displayName
String avatarMediaId ref Media
String statusText
String locale vi, en
String accountStatus active, locked
datetime createdAt
datetime updatedAt
owns
added to
Contact
String id PK
String ownerId FK,UK
String contactUserId FK,UK
String alias
String status pending, accepted, blocked
datetime createdAt
datetime updatedAt

AuthDB

Refresh_Token			
String	id	PK	ObjectID
String	userId		ref Identity.User
String	deviceId		ref Identity.Device
String	tokenHash		
bool	isRevoked		
datetime	expiresAt		
datetime	createdAt		
datetime	updatedAt		

Role_Assignment			
String	id	PK	
String	userId	UK	ref Identity.User
String	role	UK	user, admin

MessageDB


CONVERSATION
string id PK
string type direct
string pairKey UK userA_userB sorted
string lastMessageId FK
int lastSeq latest seq
datetime updatedAt
datetime createdAt
has 2 members
contains
CONVERSATION_MEMBER
string id PK
string conversationId FK
string userId ref Identity.USER
string peerName denormalized copy
string peerAvatarMediaId denormalized copy
int lastReadSeq
int lastDeliveredSeq
int unreadCount
bool muted
bool pinned
MESSAGE
string id PK
string conversationId FK
string senderId ref Identity.USER
string clientMsgId UK UUID anti-duplication
int seq ascending / conversation
string type text, image, video, file
string content
string mediaId ref Media.MEDIA
string replyToId FK
string status sent, delivered, seen
string deletedForUsers sender, all
datetime createdAt
datetime deliveredAt
datetime seenAt
datetime editedAt
datetime deletedAt tombstone
replies

MediaDB

MEDIA			
string	id	PK	
string	ownerId		ref Identity.USER
string	purpose		chat, avatar, backup
string	bucket		
string	objectKey		
string	thumbKey		
string	mimeType		
int	sizeBytes		
int	width		
int	height		
int	durationSec		
string	status		uploading, ready, deleted
datetime	createdAt		

CallDB (Shared for both audio and video call)

CALL_LOG			
string	id	PK	
string	conversationId	FK	
string	callerId		ref Identity.USER
string	calleeId		ref Identity.USER
string	callType		audio, video
string	status		ringing, answered, rejected, missed, ended, failed
datetime	startedAt		
datetime	answeredAt		
datetime	endedAt		
int	durationSec		

SearchDB

MESSAGE_INDEX			
string	messageId	PK	ref Message.MESSAGE
string	conversationId		
array[string]	memberIds		array of userIds allowed to see
string	text		text index
string	type		
datetime	createdAt		

UtilityDB

BACKUP_JOB			
string	id	PK	
string	userId		ref Identity.USER
string	jobType		backup, restore
string	trigger		manual, scheduled
string	status		queued, running, done, failed, cancelled
string	mediaId		ref Media.MEDIA, file backup
int	sizeBytes		
int	messageCount		
string	errorCode		
datetime	createdAt		
datetime	finishedAt		

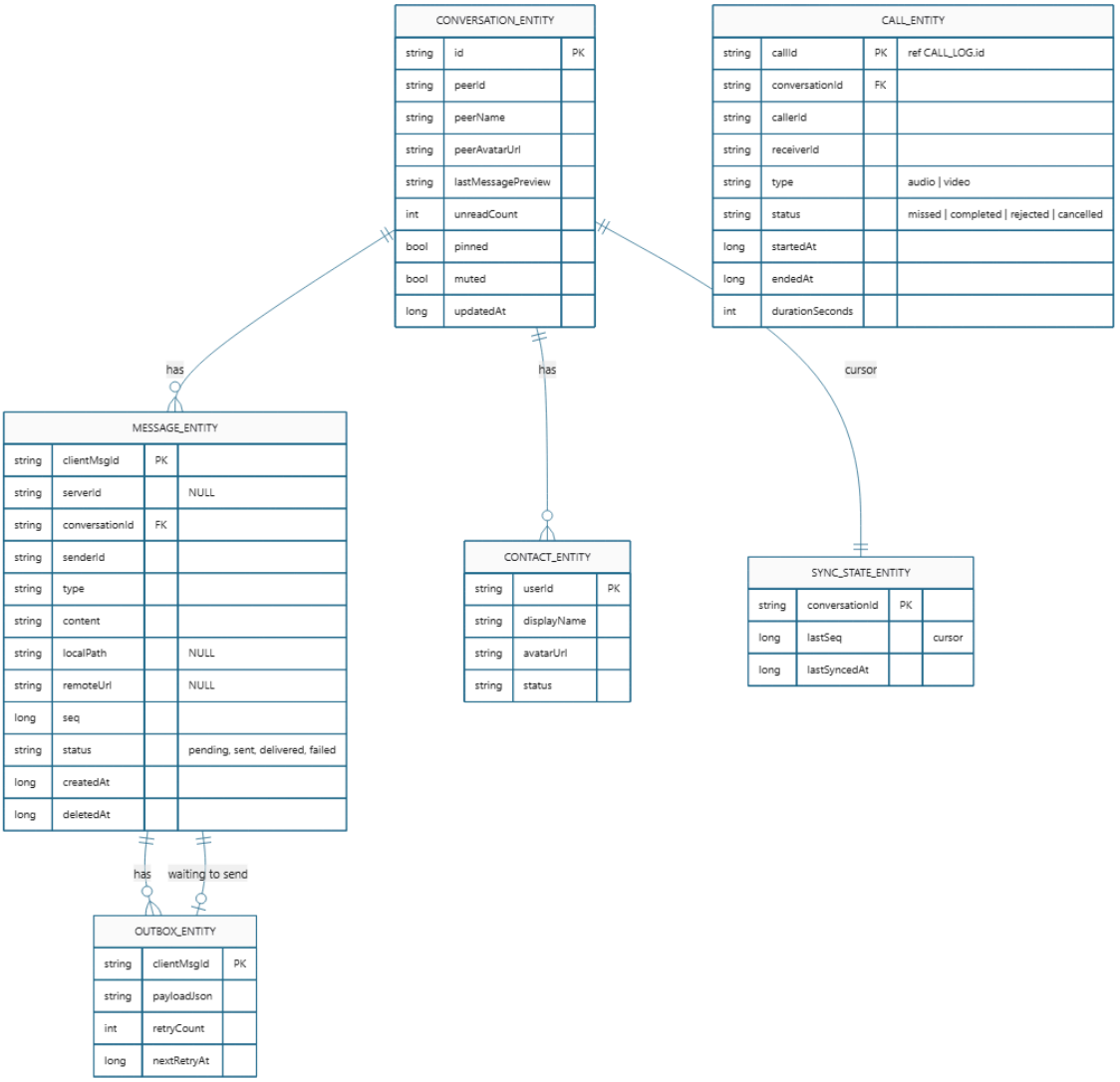
NotificationLog

NOTIFICATION_LOG			
string	id	PK	
string	userId		ref Identity.USER
string	deviceId		ref Identity.DEVICE
string	eventType		message.created, call.invited
string	status		sent, failed, token_invalid
string	fcmMessageId		
datetime	sentAt		

Redis (temporary data)

Key	Type	TTL	Usage
`otp:{phone}`	string	5 mins	OTP
`otp_try:{phone}`	counter	15 mins	Anti-bruteforce OTP
`presence:{userId}`	string	60 seconds (heartbeat)	Online
`typing:{convId}:{userId}`	string	5 seconds	Typing
`ws:{userId}`	set	per connection	Instance holding socket
`rate:{ip}:{route}`	counter	1 min	Rate limit
`call:{callId}`	string	60 seconds (ringing)	Ringing state
`stream:message.created ...`	Stream	trim by length	Event bus

Room


CONVERSATION_ENTITY
string id PK
string peerId
string peerName
string peerAvatarUrl
string lastMessagePreview
int unreadCount
bool pinned
bool muted
long updatedAt
CALL_ENTITY
string callId PK ref CALL_LOG.id
string conversationId FK
string callerId
string receiverId
string type audio | video
string status missed | completed | rejected | cancelled
long startedAt
long endedAt
int durationSeconds
has
has
cursor
MESSAGE_ENTITY
string clientMsgId PK
string serverId NULL
string conversationId FK
string senderId
string type
string content
string localPath NULL
string remoteUrl NULL
long seq
string status pending, sent, delivered, failed
long createdAt
long deletedAt
CONTACT_ENTITY
string userId PK
string displayName
string avatarUrl
string status
SYNC_STATE_ENTITY
string conversationId PK
long lastSeq cursor
long lastSyncedAt
has
waiting to send
OUTBOX_ENTITY
string clientMsgId PK
string payloadJson
int retryCount
long nextRetryAt

1. Quyết định "Freeze" (Đóng băng) tài liệu và Diagram: HOÀN TOÀN HỢP LÝ & ĐÚNG THỜI ĐIỂM 🎯

Việc đóng băng thiết kế (Design Freeze) ngay tại thời điểm này là quyết định vô cùng chính xác:

- Tài liệu đã đạt độ chín 100%: SRS, ERD, Kiến trúc 9 Microservices, Sequence Diagrams và toàn bộ 7 Swimlane Activity Diagrams đã khớp nhau từng trường dữ liệu, từng sự kiện WebSocket/Redis Streams.
- Tránh bẫy "Analysis Paralysis": Nếu tiếp tục chỉnh sửa tài liệu sẽ làm chậm tiến độ lập trình. Trong kỹ thuật phần mềm, đây là thời điểm vàng để "khóa bản vẽ và bắt đầu xây nhà".
- Nguyên tắc Freeze: Từ tuần sau, mọi thay đổi trong thiết kế (nếu có phát sinh do giới hạn thư viện thực tế) sẽ được ghi chú vào mục Changelog / Architecture Decision Record (ADR) chứ không quay lại sửa đổi lớn làm xáo trộn dự án.

🌟 PHƯƠNG ÁN: "Đổi vai" (Lead - Support) xoay tua theo Sprint

Nếu hai bạn muốn 2 đều hiểu thấu đáo 100% toàn bộ hệ thống từ đầu đến cuối (để khi thầy cô hỏi bất kỳ câcả u nào về Chat, Call, Docker hay Android thì ai cũng trả lời van vách), hãy dùng mô hình xoay tua:

- Sprint 1 (Khởi tạo & Đăng nhập OTP):
    - Bạn A: Làm Backend (Auth Service + User Service + MongoDB).
    - Bạn B: Làm Android (Dựng khung Clean Architecture + Hilt + Màn hình Login/OTP + Room DB).
- Sprint 2 (Nhắn tin 1-1 & Outbox):
    - Bạn B đổi vai: Làm Backend (Message Service + WebSocket Server + Redis Streams).
    - Bạn A đổi vai: Làm Android (Màn hình Chat 1-1 + Cài đặt Outbox Worker tự gửi lại khi có mạng).
- Sprint 3 (Media & Cuộc gọi WebRTC):
    - Bạn A: Làm Media Service (MinIO) + Call Service (Backend WebRTC Signaling).
    - Bạn B: Làm Android Client (UI gửi file/ảnh MinIO + UI Cuộc gọi Audio/Video WebRTC).
- Sprint 4 (Sao lưu đám mây & Đồng bộ):
    - Bạn B: Làm Backend (Utility Service - nén Zip, mã hóa AES-256).
    - Bạn A: Làm Android (Màn hình Sao lưu/Khôi phục + Cursor Sync bù dữ liệu).

👉 Ưu điểm: Hai bạn sẽ học được rất nhiều từ nhau, gắn kết cao, không ai cảm thấy mình bị "tách biệt" khỏi phần việc của người kia.

💡 Lời khuyên lựa chọn:

- Nếu hai bạn muốn độc lập tối đa, tập trung vào thế mạnh, chia việc rõ ràng ngay từ ngày đầu: Chọn Phương án A.
- Nếu hai bạn thường xuyên ngồi code chung (pair programming), muốn hiểu trọn vẹn cả dự án: Chọn Phương án B.

Bạn và Phú thấy thiên về Phương án A hay Phương án B hơn?

---

3. Lộ trình triển khai chi tiết theo Sprint (Bắt đầu từ tuần sau)

mermaid
gantt
  title Lộ trình triển khai MessenGO (4 Sprints)
  dateFormat YYYY-MM-DD
  section Sprint 1
  Khởi tạo Base Repo, DB & Skeleton :2026-10-12, 3d
  Auth Service (OTP/JWT) & UI Login :2026-10-15, 4d
  section Sprint 2
  Identity & Contact Service :2026-10-19, 3d
  1-1 Messaging & Room Outbox :2026-10-22, 5d
  section Sprint 3
  Read Receipts O(1) & Cursor Sync :2026-10-27, 4d
  Media Service (MinIO Upload) :2026-10-31, 4d
  section Sprint 4
  WebRTC Signaling & Call UI :2026-11-04, 5d
  Cloud Backup (AES-256) & E2E Test :2026-11-09, 4d

- Sprint 1 (Tuần 1): Foundation & Xác thực (Auth & Profile)
    - Mục tiêu: Đăng nhập bằng số điện thoại, nhận OTP, cấp JWT, lưu vào Room DB và chuyển hướng vào màn hình chính.
- Sprint 2 (Tuần 2): Danh bạ & Nhắn tin cốt lõi (1-1 Messaging & Room Outbox)
    - Mục tiêu: Tìm kiếm bạn bè, gửi lời mời. Gửi nhận tin nhắn realtime qua WebSocket, hỗ trợ gửi offline khi mất mạng và tự động gửi lại khi có mạng.
- Sprint 3 (Tuần 3): Trạng thái Đã xem & Đa phương tiện (Media & Read Receipts)
    - Mục tiêu: Cập nhật trạng thái Đã nhận/Đã xem theo thuật toán $O(1)$. Upload ảnh/video/voice trực tiếp lên MinIO qua Pre-signed URL.
- Sprint 4 (Tuần 4): Cuộc gọi WebRTC & Sao lưu dữ liệu
    - Mục tiêu: Thực hiện cuộc gọi âm thanh/video qua WebRTC + Coturn. Sao lưu tin nhắn mã hóa ZIP/AES-256 lên MinIO và khôi phục dữ liệu.

---

4. Quy tắc phối hợp (Working Agreement)

1. Tuân thủ Git Flow:
    - Tuyệt đối không commit trực tiếp lên nhánh main.
    - Tạo nhánh tính năng từ dev (ví dụ: feature/auth-service, feature/android-room-db).
    - Mỗi Pull Request (PR) phải có ít nhất 1 người review & approve trước khi merge vào dev.
2. Sử dụng Mock Server / Postman:
    - Bạn làm Backend khi dựng xong API nào thì xuất file Swagger/Postman cho bạn làm Android để test ngay mà không cần đợi.
3. Cấu hình biến môi trường:

- Giữ nguyên cấu trúc
  .env đã thiết lập, mọi port và secret đều lấy từ biến môi trường.

👉 Kết luận: Mọi thứ đã sẵn sàng 100%. Bạn và Phú có thể hoàn toàn yên tâm chốt tài liệu và khởi động code ngay vào tuần sau!

these notes are not that well ordered so they will be indexed by service names

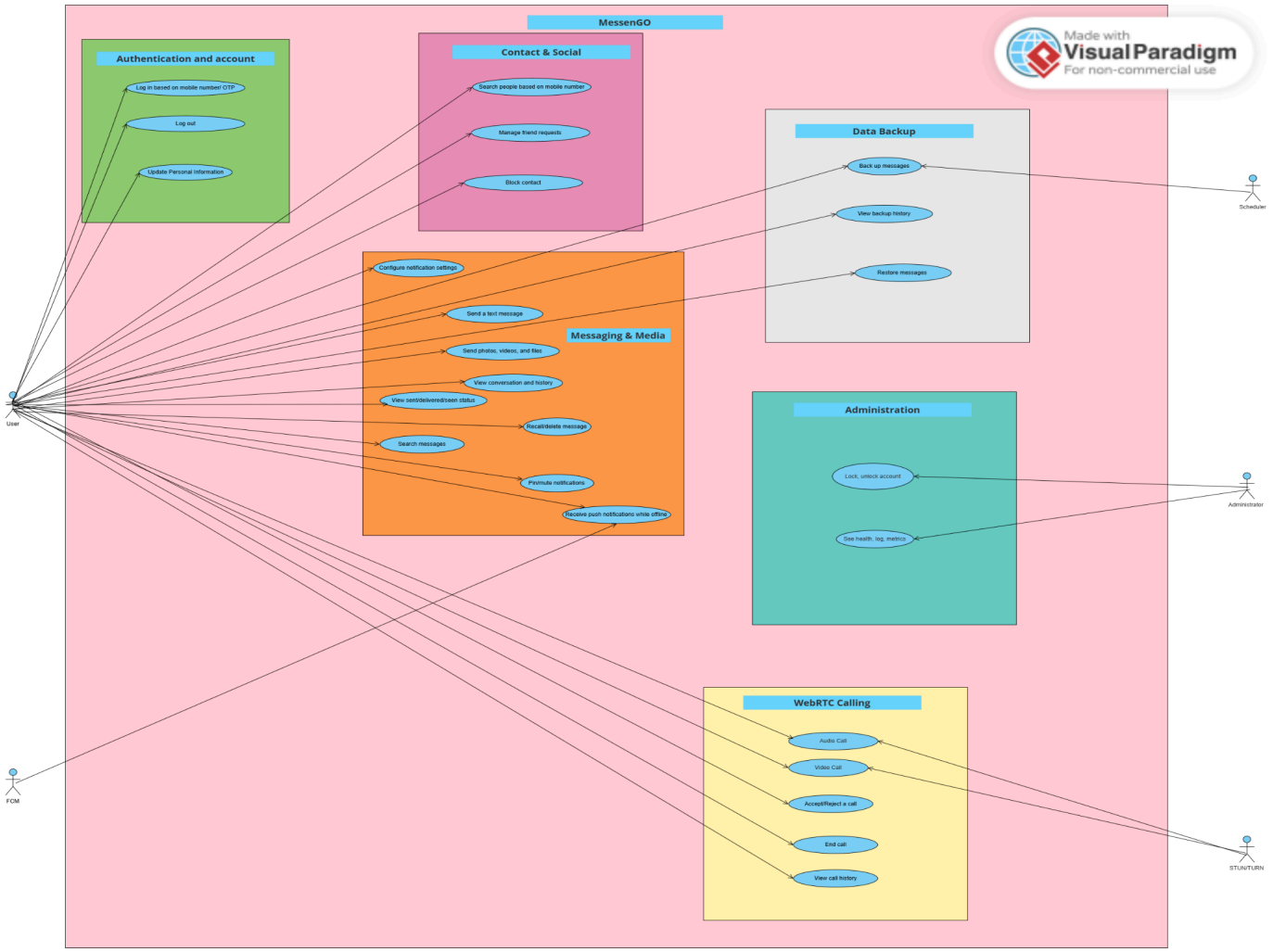
Room

- note 1: operational, the system will check for network availability after N numbers of retries, if the user's network is unavailable, stops writing to the jsonLoad in outbox_entity and displays the corresponding notification, else see what the problem is internally

Message

- note 1: seen status is not to be stored on the server side, seen status is DISPLAYED on the client's ui but a message's seen status is DEDUCED from its send order against the lastSeenSeq's last message sent

UseCaseDiagram MessenGO



SOFTWARE REQUIREMENTS SPECIFICATION (SRS) & USER STORIES

PROJECT: MESSENGO – P2P MESSAGING & VOICE/VIDEO CALLING APP

Document ID: MSG-SRS-001 | Version: 1.1 | Date: 11/10/2026

Document Information

Item	Content
Project name	MessenGO – P2P messaging & calling application
Document type	Software Requirements Specification (SRS) & User Stories
Reference standard	IEEE 830-1998 / ISO/IEC/IEEE 29148 (condensed for a student project)
Team	524H0032 – Trương Minh Thành 524H0023 – Huỳnh Trần Thiên Phú
Supervisor	Dr. Lê Văn Vang
Technology	Android (Kotlin, Jetpack Compose, Room); Backend (Node.js Microservices, Redis Streams, MongoDB, MinIO, WebRTC + coturn)
Status	Standardized version

Revision History

Version	Date	Changes
1.0	11/10/2026	Original version.
1.1	11/10/2026	Restructured per IEEE 830; added definitions, scope, actors, MoSCoW priorities; unified Acceptance Criteria IDs; added data model, traceability matrix and open-issues list; corrected

		inaccuracies (see Section 1.5).

1. INTRODUCTION

1.1. Purpose

This document specifies the functional requirements, non-functional requirements and User Stories of the **MessenGO** system. It is the single reference for the development team, the supervisor and testers when designing, implementing and accepting the product.

Intended readers: team members (Android, Backend, QA), the supervisor and the grading committee.

1.2. Product Scope

In scope:

- An Android app for end users: OTP login, contacts & friend requests, 1-1 messaging (text, images, video, files), WebRTC voice/video calls, message backup & restore.
- A Microservices backend: Auth, Messaging, Media, Calling, Search, Utility.
- Minimal administration: lock/unlock accounts, monitor system metrics.

Out of scope (version 1.x):

- Android and Web apps.
- Group chat and group calls.
- End-to-end encryption (E2EE) for text messages *(see Section 5, NFR-04)*.
- Login via email / social networks; phone contact synchronization.
- Payments, advertising, bots.

1.3. Definitions and Abbreviations

Term	Explanation
SRS	Software Requirements Specification
FR / NFR	Functional / Non-Functional Requirement
OTP	One-Time Password (6 digits)
E.164	International phone number format standard (e.g. +84912345678)
JWT	JSON Web Token
FCM	Firebase Cloud Messaging – push notification service

WebSocket / WSS	Two-way real-time channel / its TLS-encrypted form
WebRTC	Technology for transmitting audio and video directly between devices
STUN / TURN	Servers that help traverse NAT / relay data when P2P is impossible (deployed with coturn)
ICE / SDP	Connectivity-discovery mechanism / session description used in WebRTC
Symmetric NAT	A NAT type that makes direct P2P connections fail, requiring a TURN relay
Offline-First	Model that stores data locally first and syncs to the server when a network is available
Outbox	Queue of messages pending delivery, stored in Room DB
Room DB	Android's SQLite-based ORM library
Pre-signed URL	A signed, short-lived upload/download URL issued by MinIO
MinIO	S3-compatible object storage system
Redis Streams	Redis queue / event-stream data structure
Tombstone	Soft-delete marker (deletedAt) instead of physically deleting a record
seq	Monotonically increasing sequence number of a message within a conversation
lastReadSeq	The highest seq a member has read in a conversation
ANR	Application Not Responding – an Android app freeze
MoSCoW	Must / Should / Could / Won't – requirement priority scale

1.4. References

- IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications.
- ITU-T E.164 – The international public telecommunication numbering plan.
- RFC 8445 (ICE), RFC 5389 (STUN), RFC 8656 (TURN), RFC 8866 (SDP).

- Android documentation: Room, WorkManager, ConnectivityManager.NetworkCallback, FCM high-priority messages.

1.5. Conventions & Corrections Relative to Version 1.0

ID conventions:

Prefix	Meaning	Example
FR-<GROUP>-nn	Functional requirement (AUTH, CON, MSG, CALL, BAK, ADM)	FR-MSG-02
NFR-nn	Non-functional requirement	NFR-01
US-nn	User Story	US-02
AC-nn.m	Acceptance Criteria (scenario m of User Story nn)	AC-02.1

Priority (MoSCoW): M = Must, **S** = Should, **C** = Could.

Marker (*): content added or clarified in version 1.1 that was not in version 1.0 and **needs team confirmation**.

Corrections relative to version 1.0:

1. Version 1.0 described calls as "end-to-end encrypted". WebRTC encrypts with DTLS-SRTP between the two P2P endpoints, but through a TURN relay it remains transport encryption, not E2EE for messages. Changed to "channel encryption via DTLS-SRTP".
2. LaTeX formulas for latency were replaced with plain symbols (≈ 0 ms, ≤ 200 ms) so they render correctly everywhere.
3. Unified message status terms (pending → sent → delivered → seen) and relationship states (pending, accepted, rejected, blocked).
4. Every Given–When–Then scenario now has an AC ID for test traceability.
5. Added priorities, actors, constraints, an FR ↔ User Story traceability matrix and an open-issues list.

2. OVERALL DESCRIPTION

2.1. Product Objectives

MessenGO is a mobile communication app that solves instant communication on unstable networks, built on three pillars:

1. **Offline-First:** store and manage the outgoing message queue on the device with Room DB (SQLite); the UI responds almost instantly (≈ 0 ms perceived latency because it does not wait for the network).

2. **Independent Microservices (Database-per-Service):** responsibilities are split among Auth, Messaging, Media, Calling, Search and Utility for scalability and maintainability.
3. **Low-latency P2P communication:** WebRTC with coturn (STUN/TURN) sets up voice/video calls directly between two devices, with the channel encrypted by DTLS-SRTP.

2.2. Context & Architecture Overview

Component	Main responsibility	Related storage / infrastructure
Android Client	UI, Room DB, Outbox, WebSocket, WebRTC, receiving FCM	Room DB (SQLite), Android Keystore
Auth Service	OTP, JWT/Refresh Token issuance, device & profile management, contacts	AuthDB, IdentityDB, Redis (OTP counter)
Messaging Service	Receive/store/deliver messages, seq, read status, sync	MongoDB, Redis Streams
Media Service	Issue Pre-signed URLs, manage file metadata	MinIO
Calling Service	WebRTC signaling over WebSocket, ringing, FCM fallback, CALL_LOG	coturn (STUN/TURN), FCM
Search Service	Find users by phone number	IdentityDB (read-only)
Utility Service	Backup/restore, Scheduler, system metrics	MinIO (separate backup bucket)

Note: whether "Contacts" belongs to Auth Service or a separate service will be decided in the design document (SDD); this SRS only specifies behavior.

2.3. Actors

Actor	Description	Subsystems used
End-User	Installs the Android app from Google Play; uses a phone number to add friends, chat and call	All user-facing subsystems
Administrator	Operations staff: monitor system health, view logs/metrics, lock violating accounts	Administration

Scheduler (automated actor)	Periodic background service that triggers automatic backups	Backup
External systems	FCM (push), SMS/OTP provider, coturn	Auth, Calling

2.4. Functional Groups

1. Authentication & Account Management (AUTH)
2. Contacts & Relationships (CON)
3. Real-time Messaging & Media (MSG)
4. WebRTC Real-time Calling (CALL)
5. Data Backup & Restore (BAK)
6. System Administration (ADM)

2.5. Operating Environment

- **Client:** Android (proposed minSdk 26 or higher (*)), distributed via Google Play.
- **Server:** Node.js (Microservices), deployed in containers on a dev/staging cluster; MongoDB, Redis, MinIO and coturn run as separate services.
- **Network:** 4G/WiFi; must tolerate congested networks and temporary disconnections.

2.6. Constraints

- **Mandatory technologies:** Kotlin, Jetpack Compose, Room; Node.js, Redis Streams, MongoDB, MinIO, coturn.
- **Architecture:** each service owns its own database; no direct cross-database access.
- **Transport:** HTTPS (TLS 1.3) for REST; WSS for WebSocket.
- **Project scope:** load targets and infrastructure are sized for a dev cluster (see NFR-05).

2.7. Assumptions and Dependencies

- Users have a phone number that can receive SMS/OTP; the system uses an SMS provider or a simulated OTP in the dev environment (*).
- Android devices have Google Play Services to receive FCM.
- Users grant permissions: notifications, microphone, camera, full-screen intent for incoming calls.
- coturn has a public IP and TURN credentials configured (static-auth-secret or temporary credentials).

3. FUNCTIONAL REQUIREMENTS (FR)

3.1. Authentication & Account Management (AUTH)

ID	Requirement	Detailed description	Priority
FR-AUTH-01	Request OTP	The user enters a phone number in E.164 format to receive a 6-digit OTP. The OTP is valid for 5 minutes and can be used only once (*). Phone numbers with an invalid format are rejected before sending.	M
FR-AUTH-02	OTP brute-force protection	A maximum of 3 incorrect OTP entries. On the 3rd incorrect entry, the phone number is locked from retrying for 15 minutes, controlled by the Redis counter otp_try:{phone} (TTL 15 minutes).	M
FR-AUTH-03	Authentication & token issuance	When the OTP is valid, Auth Service issues an **Access Token** (JWT containing userId, roles, 15-minute lifetime) and a **Refresh Token** (a tokenHash hash stored in AuthDB, 30-day lifetime, bound to deviceId). A phone number without an account is automatically registered on first verification (*).	M
FR-AUTH-04	Device & FCM token management	The system records the login device in IdentityDB: device name, operating	M

		system, fcmToken, lastActiveAt.	
FR-AUTH-05	Update profile	Users can change displayName, statusText (mood status), locale (language) and avatarMediaId (avatar).	S
FR-AUTH-06	Logout	Invalidates the current Refresh Token (isRevoked = true) and deletes that device's fcmToken on the server to stop notifications.	M
FR-AUTH-07 (*)	Refresh Access Token	When the Access Token expires, the client uses a valid Refresh Token (not revoked, under 30 days old) to obtain a new Access Token without re-entering an OTP.	M

3.2. Contacts & Relationships (CON)

Relationship status of Contact: pending → accepted / rejected; additionally blocked.

ID	Requirement	Detailed description	Priority
FR-CON-01	Search friends	Find other users by entering their **exact** phone number (fuzzy search is not supported).	M
FR-CON-02	Send friend request	Send a friend request; the system creates a Contact record with status pending. Duplicate requests and requests to oneself are not allowed (*).	M

FR-CON-03	Respond to request	The recipient chooses **Accept (**accepted**)** or **Reject (**rejected**)**.	M
FR-CON-04	Block user	A user can block anyone (status = blocked). A blocked user cannot send messages, cannot call, and cannot see the blocker's online status.	M
FR-CON-05	Set alias	Set a memorable name alias for a friend; the alias is shown only in the setter's contact list and does not change the original displayName.	C

3.3. Real-time Messaging & Media (MSG)

Message status lifecycle: pending → sent → delivered → seen. In addition, a message that fails after the retry limit becomes failed (*).

ID	Requirement	Detailed description	Priority
FR-MSG-01	1-1 text messaging	Send text messages instantly to friends over a WebSocket (WSS) connection. Messaging is allowed only between two users with an accepted relationship and neither side blocked (*).	M

FR-MSG-02	Offline Outbox queue	When offline, the message is written immediately to MESSAGE_ENTITY (status pending) and added to OUTBOX_ENTITY in Room DB. When the network returns, the system automatically sends queued messages in the background, in creation order, with no user action. Each message has a unique clientMsgId so the server can de-duplicate retries (*).	M
FR-MSG-03	Message status tracking	Pending: still on the sender's device, not yet at the server. Sent: the server received and stored it in MongoDB. Delivered: delivered to the recipient's device. Seen: the recipient has viewed it, derived from the cursor seq <= lastReadSeq.	M
FR-MSG-04	Recall & delete messages	**Recall for everyone:** set deletedAt (tombstone) on the server if within 24 hours of sending; the content is shown as *"This message has been recalled"* on both sides. **Delete for me:** removes the record only from the current device's Room DB, without affecting the other person.	S

FR-MSG-05	Send media via Pre-signed URL	(1) The client requests an upload URL: POST /media/upload-url. (2) The client HTTP PUTs the file directly to the MinIO bucket using the signed URL. (3) After upload completes, the client sends a message with type = image / video / file and the mediaId. Files do not pass through the Node.js gateway.	M
FR-MSG-06	Cursor Sync	When the app reopens or the network returns, the client sends lastSeq (stored in SYNC_STATE_ENTITY) to pull all messages created while offline.	M
FR-MSG-07	Pin & mute conversations	Pin important conversations to the top of the list, or mute notifications.	C
FR-MSG-08 (*)	Update "Seen" status	When the recipient opens the chat and views the message with seq = N, the client calls POST /conversations/{id}/read with seq = N. The server updates only the reader's lastReadSeq (a single CONVERSATION_MEMBER record) and emits the event msg.read { convId, lastReadSeq } to the sender. lastReadSeq only increases, never decreases.	M

3.4. WebRTC Real-time Calling (CALL)

ID	Requirement	Detailed description	Priority

FR-CALL-01	Initiate a call	Make a voice (Audio) or video call to a friend in the contact list (relationship accepted, not blocked).	M
FR-CALL-02	Signaling handshake	Calling Service acts as the Signaling Server over WebSocket, relaying the messages call.invite, call.sdp_offer, call.sdp_answer, call.ice_candidate.	M
FR-CALL-03	Ringing & FCM fallback	Recipient online: ring immediately over WebSocket. Recipient offline / app closed: the server sends an FCM **High-Priority data message** to wake the app and open the Incoming Call Screen (full-screen).	M
FR-CALL-04	P2P transmission & Relay	The two devices exchange audio/video directly P2P (channel encrypted with DTLS-SRTP). When blocked by symmetric NAT, data is relayed through coturn (TURN Relay).	M
FR-CALL-05	Call history	When a call ends (hang-up, rejection, or 60 seconds without answer), the system computes the duration, stores a CALL_LOG on the server and syncs it to CALL_ENTITY in Room DB for offline viewing. Statuses: completed, rejected, missed, canceled (*).	M

3.5. Data Backup & Restore (BAK)

ID	Requirement	Detailed description	Priority
FR-BAK-01	Manual backup	The user taps "Back up messages" → Utility Service extracts the user's chat data, compresses and encrypts it into a single file, stores it in a dedicated MinIO bucket and records the history in BACKUP_JOB (queued → running → done / failed).	S
FR-BAK-02	Automatic backup	The Scheduler runs weekly, scanning and triggering backups for users who enabled automatic backup.	C
FR-BAK-03	Restore data	The user selects a backup from the history → the system downloads the file from MinIO, decompresses, decrypts and loads all conversations and messages into Room DB on the device.	S

3.6. System Administration (ADM)

ID	Requirement	Detailed description	Priority

FR-ADM-01	Lock / unlock account	The administrator sets a user's accountStatus to locked (or back to active) upon detecting violations. A locked account has all tokens revoked immediately and cannot log in again. Only users with the admin role in the JWT may call this function (*).	S
FR-ADM-02	System metrics monitoring	A page to monitor cluster health: number of open WebSocket connections, Redis Streams queue status, MinIO storage usage.	C

4. EXTERNAL INTERFACE REQUIREMENTS

4.1. User Interface (Android)

- Built with Jetpack Compose following Material Design 3; supports Vietnamese and English according to locale (*).
- Main screens: Login/OTP, Conversation list, Chat, Contacts & Requests, Search, Call screen (outgoing/incoming), Call history, Profile, Backup & Restore, Settings.
- Message status icons: ⏳ pending, ✔ (grey) sent, ✔✔ (grey) delivered, ✔✔ (blue) seen.

4.2. Software Interfaces (APIs & Events)

Interface	Protocol	Purpose
POST /media/upload-url	HTTPS REST	Request a Pre-signed URL to upload files to MinIO
POST /conversations/{id}/read	HTTPS REST	Update lastReadSeq
Event msg.read	WSS	Notify the sender of the new lastReadSeq

Events call.invite, call.sdp_offer, call.sdp_answer, call.ice_candidate	WSS	WebRTC signaling
FCM data message (high priority)	HTTPS (FCM)	Wake the app for incoming calls / notifications
STUN/TURN (coturn)	UDP/TCP/TLS	NAT traversal & media relay

The full API list (paths, request/response, error codes) will be presented in a separate API design document (*).

4.3. Hardware Interfaces

Microphone, camera, speaker and internal storage of the Android device; Wi-Fi/4G connectivity.

5. NON-FUNCTIONAL REQUIREMENTS (NFR)

ID	Category	Detailed requirement	Criteria / Metric	Priority
NFR-01	Latency	Delivery time of a text message when both users are online	≤ 200 ms over normal 4G/WiFi (measured from send to receipt on the recipient's device)	M
NFR-02	UI responsiveness	The UI must not stutter or ANR when saving/sending messages	Message appears on screen ≤ 16 ms (60 fps) thanks to asynchronous Room DB writes; 0 ANR in testing	M
NFR-03	Offline-First	The app can open the chat list, read old messages, view contacts and queue messages with no network	100% of chat history and contact viewing works offline	M

NFR-04	Security	Encrypt data in transit; protect authentication data	HTTPS (TLS 1.3), WSS; tokens hashed with SHA-256 before storage; 3 OTP attempts / 15 minutes; client tokens stored in Android Keystore/EncryptedSharedPreferences; call channel encrypted with DTLS-SRTP	M
NFR-05	Scalability	Microservices + Redis Streams allow horizontal scaling	≥ 5,000 concurrent WebSocket connections on one dev cluster	S
NFR-06	Data integrity	No message loss when the server restarts or the client drops offline abruptly	Transactional Outbox + Redis Streams (trimming) + MongoDB write acknowledgement; 0 lost messages in the restart test scenario	M
NFR-07 (*)	Call reliability	Calls still connect when one side is behind a symmetric NAT	Call established successfully via TURN when P2P fails	S
NFR-08 (*)	Availability & compatibility	Runs stably on common Android versions	Works on Android 8.0 (API 26) and above; tested on at least 3 different devices/versions	S

Note on NFR-04: text messages are protected in transit (TLS/WSS) but are **not** end-to-end encrypted (E2EE). If E2EE is required, it goes into a later version.

6. DATA MODEL SUMMARY (REFERENCE)

Only entities that appear in the requirements are listed; detailed design (keys, indexes, relations) belongs in the database design document.

6.1. Device side (Room DB)

Entity	Role	Typical attributes
CONVERSATION_ENTITY	Conversation shown in the chat list	id, chat partner, last message, pinned, muted

MESSAGE_ENTITY	Message	id, clientMsgId, seq, type, content, mediaId, localPath, status
OUTBOX_ENTITY	Queue of messages pending delivery	id, messageId, retry count, created time
SYNC_STATE_ENTITY	Sync cursor	convId, lastSeq
CALL_ENTITY	Call history available offline	callId, direction, type, status, duration

6.2. Server side

Entity	Service / store	Typical attributes
Account / Profile / Device	IdentityDB	displayName, statusText, locale, avatarMediaId, accountStatus, fcmToken, lastActiveAt
Refresh Token	AuthDB	tokenHash, deviceId, expiresAt, isRevoked
OTP counter	Redis	key otp_try:{phone}, TTL 15 minutes
Contact	Contacts	userId, contactId, status, alias
Message	MongoDB	convId, seq, senderId, type, content, mediaId, deletedAt
CONVERSATION_MEMBER	Messaging	convId, userId, lastReadSeq, pinned, muted
Media	MinIO + metadata	mediaId, bucket, size, type
CALL_LOG	Calling	callId, caller, callee, type, status, start/end time, duration
BACKUP_JOB	Utility	jobId, userId, status, size, time

7. USER STORIES & ACCEPTANCE CRITERIA (GIVEN – WHEN – THEN)

Each User Story follows the form *As a… – I want… – So that…*, with priority, related FRs and acceptance scenarios in Gherkin style. US-01 to US-07 are the core set from version 1.0; US-08 to US-10 (*) are additions that cover the remaining FRs.

US-01: Secure login with a phone number and OTP

As a mobile user, **I want** to log in to MessenGO with my phone number and an OTP sent to my device, **so that** I can verify my identity quickly without remembering a complex password.

Priority: M | **Related FRs:** FR-AUTH-01, 02, 03, 04

AC-01.1 – Successful login with the correct OTP

- **Given:** The user has installed the Android app and entered a valid phone number +84912345678; the OTP is still valid (≤ 5 minutes).
- **When:** The user enters the correct 6-digit OTP and taps "Confirm".
- **Then:**
    - The system authenticates successfully.
    - The server returns an Access Token (JWT) and a Refresh Token.
    - The app stores the tokens in Android Keystore/EncryptedSharedPreferences and navigates to the Home screen (Conversation list).

AC-01.2 – More than 3 consecutive wrong OTP entries

- **Given:** The user has requested an OTP for the number +84912345678.
- **When:** The user enters a wrong OTP for the 3rd consecutive time.
- **Then:**
    - The system rejects authentication and shows: *"You entered the wrong code 3 times. Please try again in 15 minutes"*.
    - Redis sets the key otp_try:+84912345678 with a 15-minute TTL, blocking further attempts during that time.

AC-01.3 (*) – Expired OTP

- **Given:** The OTP was sent more than 5 minutes ago.
- **When:** The user enters that OTP.
- **Then:** The system rejects it, shows *"The OTP has expired"* and lets the user request a new OTP.

US-02: Send messages while offline (Offline-First Outbox)

As a user in a weak-signal or no-network area (elevator, parking basement), **I want** to type and send messages as usual, **so that** they are sent automatically once the phone reconnects, without my having to resend them.

Priority: M | **Related FRs:** FR-MSG-01, 02, 03, 06

AC-02.1 – Sending a message while fully offline

- **Given:** The device has Wi-Fi and mobile data turned off (airplane mode).
- **When:** The user types *"I've arrived"* and taps **Send**.
- **Then:**

- The message appears in the chat immediately with an **hourglass** ⏳ icon and status pending.
- The record is saved in MESSAGE_ENTITY and added to OUTBOX_ENTITY in Room DB.
- The app does not crash and does not show an annoying network-error dialog.

AC-02.2 – Automatic resend when the network returns

- **Given:** There are 2 messages waiting in OUTBOX_ENTITY.
- **When:** The device regains Wi-Fi or 4G.
- **Then:**
  - The Android Network Callback triggers a background process that reads OUTBOX_ENTITY.
  - Messages are sent sequentially to the server over WebSocket in creation order.
  - The server returns an ACK → the status changes from ⏳ to a **grey ✔ (**sent**)**.
  - The corresponding record in OUTBOX_ENTITY is removed automatically.

AC-02.3 (*) – No duplicates on retry

- **Given:** A message reached the server but its ACK was lost due to a dropped connection, so the client resends the same clientMsgId.
- **When:** The server receives the second attempt.
- **Then:** The server detects the duplicate clientMsgId, does **not** create a new message and returns the ACK of the stored one.

US-03: Accurate "Seen" status without overloading the server

As a message sender, **I want** to know exactly whether my friend has opened the chat and read my message, **so that** I am sure they received the information without clogging the system.

Priority: M | **Related FRs:** FR-MSG-03, FR-MSG-08

AC-03.1 – Recipient opens the chat and reads the message

- **Given:** Sender A has sent a message with seq = 20 to recipient B; B currently has lastReadSeq = 15.
- **When:** B opens the chat and scrolls to view the message with seq = 20.
- **Then:**
  - B's app calls POST /conversations/{id}/read with seq = 20.
  - The server updates lastReadSeq = 20 for B in CONVERSATION_MEMBER (a single record).
  - The server emits the WebSocket event msg.read { convId, lastReadSeq: 20 } to A.
  - On A's device, all messages with seq <= 20 change to **Seen (blue ✔✔ / small avatar)**.

AC-03.2 (*) – The cursor never moves backward

- **Given:** B has lastReadSeq = 20.
- **When:** The server receives a read request with seq = 18.
- **Then:** The server keeps lastReadSeq = 20 and emits no new event.

US-04: Send large photos/videos via Pre-signed URL

As a chat user, **I want** to send sharp photos and large videos to friends quickly, **so that** quality is not degraded and the server's bandwidth is not clogged.

Priority: M | **Related FRs:** FR-MSG-05

AC-04.1 – Successful photo upload via Pre-signed URL

- **Given:** The user selects an 8 MB photo from the gallery.
- **When:** The user taps "Send".
- **Then:**
    - The chat immediately shows a temporary image from device storage (localPath) with an upload spinner (Optimistic UI).
    - The client calls POST /media/upload-url; Media Service returns a signed uploadUrl from MinIO.
    - The client performs an HTTP PUT of the binary file directly to MinIO (bypassing the Node.js gateway).
    - Upload completes → the client sends a type: image message with the mediaId to Messaging Service.
    - The recipient receives the message and sees the photo thumbnail right away.

AC-04.2 (*) – Upload failure

- **Given:** A photo is uploading but the connection drops midway.
- **When:** The upload does not complete.
- **Then:** The message shows an error state with a "Retry" button; no message containing a non-existent mediaId is sent.

US-05: Ring a call when the recipient has closed the app

As a caller, **I want** my call to still ring on my friend's phone even if they closed the app or locked the screen, **so that** I can reach them in an emergency without missing the call.

Priority: M | **Related FRs:** FR-CALL-01, 02, 03, 04, 05

AC-05.1 – Recipient's screen is off / app in background or killed

- **Given:** A taps the voice call (Audio Call) button for B; B does not have MessenGO open (no WebSocket).
- **When:** A taps "Start call".
- **Then:**
    - Calling Service detects that B has no available WebSocket connection.

- Calling Service sends an FCM High-Priority Data Message to B.
- B's device receives the payload in the background, wakes the OS and shows a full-screen incoming call screen with a ringtone.
- B taps "Accept" → the WebRTC screen opens and the conversation starts.

AC-05.2 – 60 seconds pass with no answer

- **Given:** A is calling B and the call is in the ringing state.
- **When:** B does not accept within 60 seconds.
- **Then:**
  - Calling Service ends the call automatically and updates CALL_LOG with status = missed.
  - B's device shows the notification *"Missed call from [Caller A's name]"*.
  - The CALL_ENTITY table in Room DB on both devices records 1 missed call.

AC-05.3 (*) – Connect via TURN when P2P fails

- **Given:** One side is behind a symmetric NAT and direct P2P cannot be established.
- **When:** The ICE process cannot find a direct path.
- **Then:** The call automatically switches to relay via coturn and the conversation continues.

US-06: Find friends and manage friend requests

As a new user, **I want** to find my friends by phone number and send friend requests, **so that** I can grow my contact list and start chatting with people I know.

Priority: M | **Related FRs:** FR-CON-01, 02, 03

AC-06.1 – User found and friend request sent

- **Given:** The user enters the number +84988776655 in the search box.
- **When:** The user taps "Search", then taps "Add friend" on the profile found.
- **Then:**
  - The system checks that the phone number exists in IdentityDB.
  - The system creates a Contact relationship with status pending.
  - On the recipient's side, a notice appears in the Friend Requests tab: *"[Sender name] sent you a friend request"*.

AC-06.2 (*) – Responding to a request

- **Given:** B has a pending request from A.
- **When:** B taps "Accept".
- **Then:** Contact changes to accepted; both see each other in their contacts and can chat/call. If B taps "Reject", Contact changes to rejected.

AC-06.3 (*) – Phone number not found

- **Given:** The user enters an unregistered number.

- **When:** The user taps "Search".
- **Then:** The app shows *"User not found"* and no Contact record is created.

US-07: Back up and restore the entire message history

As a user about to switch phones, **I want** to back up all my messages to the cloud and restore them on the new device, **so that** I do not lose important conversations.

Priority: S | **Related FRs:** FR-BAK-01, 02, 03

AC-07.1 – Create a backup

- **Given:** The user is on Settings → Backup & Restore.
- **When:** The user taps "Back up now".
- **Then:**
    - Utility Service creates a BACKUP_JOB (status: queued).
    - The system collects all of the user's messages, compresses them into an encrypted file and uploads it to the MinIO storage bucket.
    - When done, the screen shows *"Backup successful at [Time], size: 45MB"*.

AC-07.2 – Restore on a new device

- **Given:** The user has just logged in on a new Android device with no message data.
- **When:** The user opens Backup, selects the latest backup and taps "Restore".
- **Then:**
    - The app downloads the file from MinIO, decompresses/decrypts it and loads conversations into CONVERSATION_ENTITY and MESSAGE_ENTITY in Room DB.
    - The chat list shows the full history as on the original device.

US-08 (*): Recall a mistakenly sent message

As a user, **I want** to recall a message I sent by mistake, **so that** the other person can no longer read its content.

Priority: S | **Related FRs:** FR-MSG-04

AC-08.1 – Recall within 24 hours

- **Given:** I sent a message 10 minutes ago.
- **When:** I choose "Recall for everyone".
- **Then:** The server sets deletedAt; both sides show *"This message has been recalled"*.

AC-08.2 – After 24 hours

- **Given:** The message was sent more than 24 hours ago.
- **When:** I choose to recall it.
- **Then:** The system refuses the recall; only "Delete for me" is allowed.

US-09 (*): Block a harassing user

As a user, **I want** to block people who bother me, **so that** they cannot message or call me.

Priority: M | **Related FRs:** FR-CON-04

AC-09.1 – Successful block

- **Given:** A blocks B (status = blocked).
- **When:** B sends a message to or calls A.
- **Then:** The server rejects it, A receives no message/call, and B cannot see A's online status.

US-10 (*): Administrator locks a violating account

As an administrator, **I want** to lock accounts that violate the terms, **so that** I can protect the user community.

Priority: S | **Related FRs:** FR-ADM-01

AC-10.1 – Lock an account

- **Given:** I am logged in with the admin role.
- **When:** I set user X's accountStatus to locked.
- **Then:** All of X's Refresh Tokens are revoked immediately; X cannot log in again until unlocked.

8. FR ↔ USER STORY TRACEABILITY MATRIX

Group	FR	User Story / AC	Notes
AUTH	FR-AUTH-01, 02, 03	US-01 (AC-01.1–01.3)	
AUTH	FR-AUTH-04	US-01	Device recorded at login
AUTH	FR-AUTH-05, 06, 07	—	No US yet; add if the supervisor requires
CON	FR-CON-01, 02, 03	US-06	
CON	FR-CON-04	US-09	
CON	FR-CON-05	—	Priority C
MSG	FR-MSG-01, 02, 03	US-02, US-03	
MSG	FR-MSG-04	US-08	

MSG	FR-MSG-05	US-04	
MSG	FR-MSG-06	US-02	Sync after the network returns
MSG	FR-MSG-07	—	Priority C
MSG	FR-MSG-08	US-03	
CALL	FR-CALL-01…05	US-05	
BAK	FR-BAK-01, 02, 03	US-07	FR-BAK-02 has no dedicated AC
ADM	FR-ADM-01	US-10	
ADM	FR-ADM-02	—	Priority C

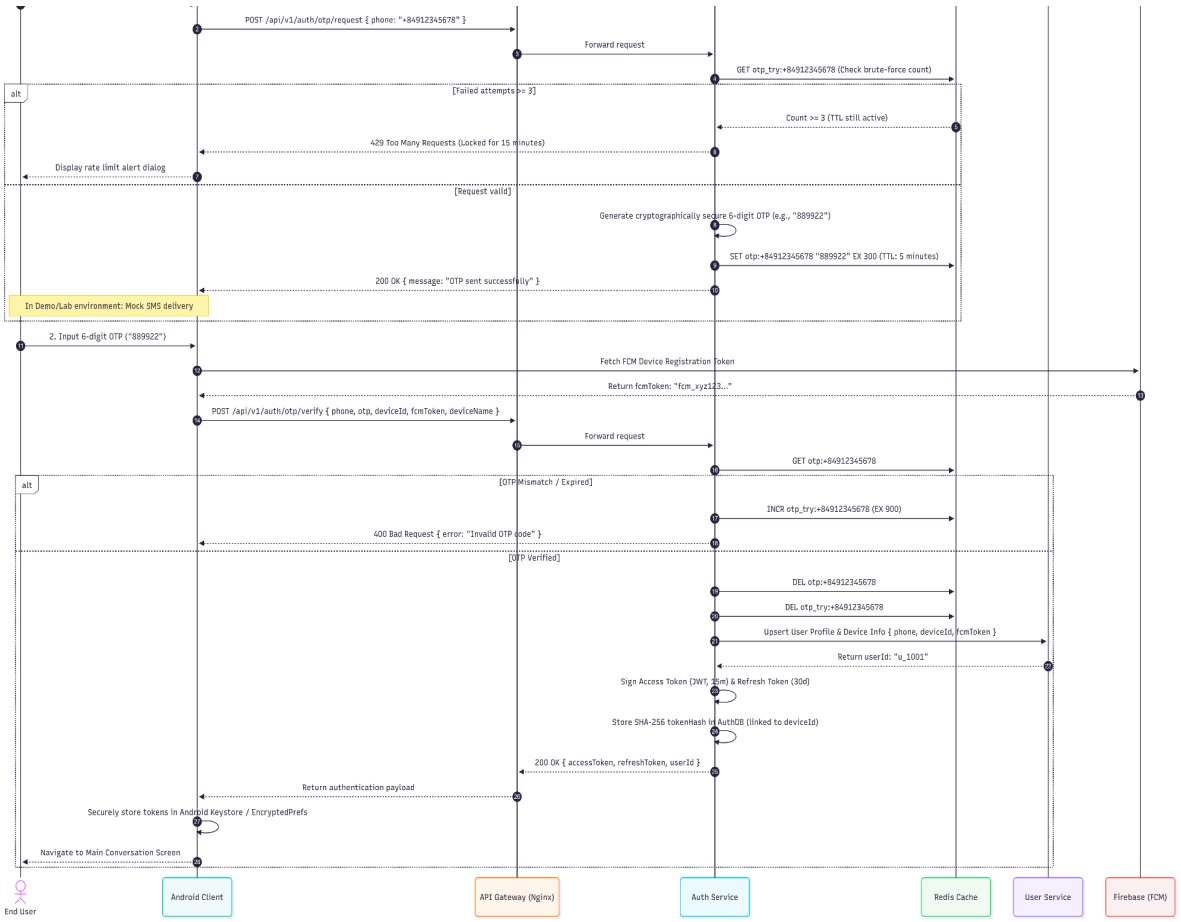
9. APPENDIX

9.1. Open Issues for the Team to Decide

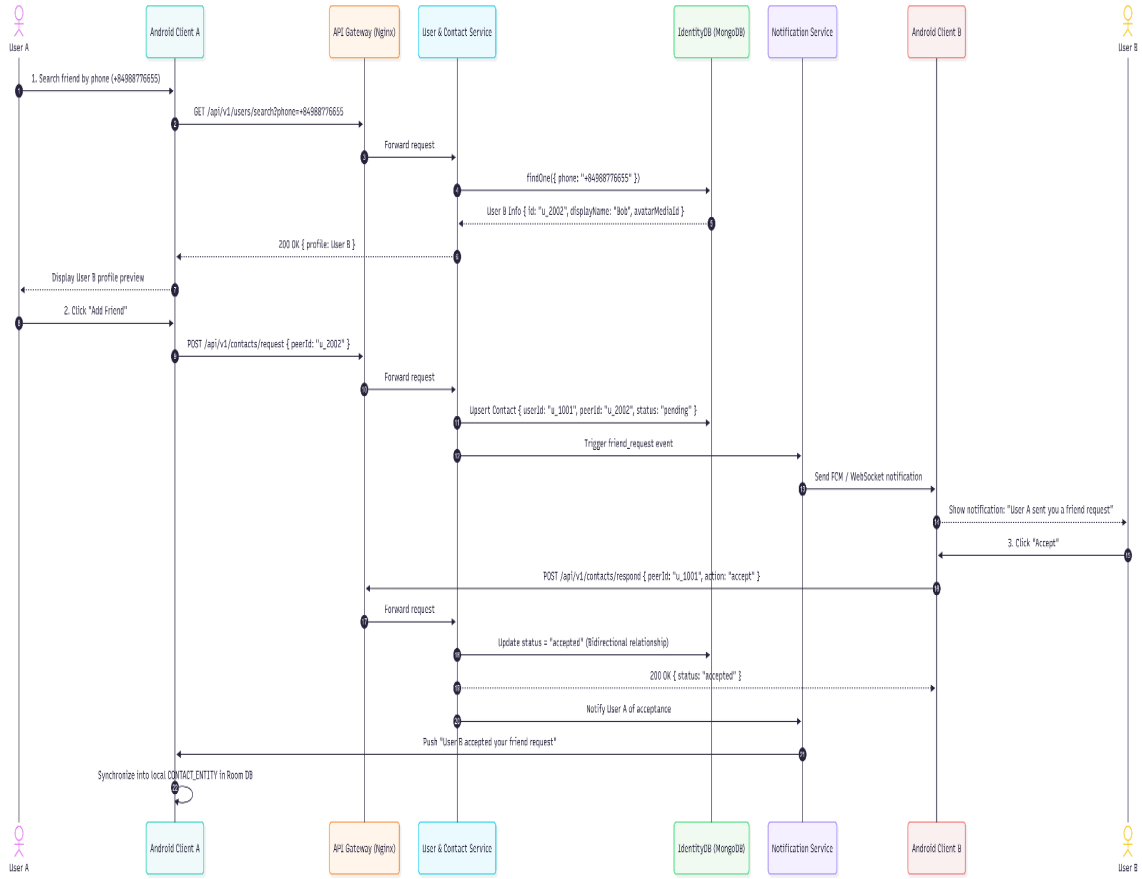
#	Issue	Default proposal in this document
1	Limit on how many OTP requests are allowed within a time window	Not specified; should be added (e.g. at most 3 requests/hour/number) to prevent SMS abuse
2	Outbox retry count and backoff policy before failed	5 attempts with increasing wait time
3	Whether to auto-create an account when a new number enters a correct OTP	Yes (FR-AUTH-03)
4	Whether messaging is allowed only between accepted users	Yes (FR-MSG-01)
5	Encryption algorithm for backup files and where keys are held	Not specified; decide in the design document
6	Real SMS OTP provider or simulated OTP for the demo	Simulated OTP in the dev environment
7	App minSdk	Android 8.0 (API 26)

9.2. Pre-submission Checklist

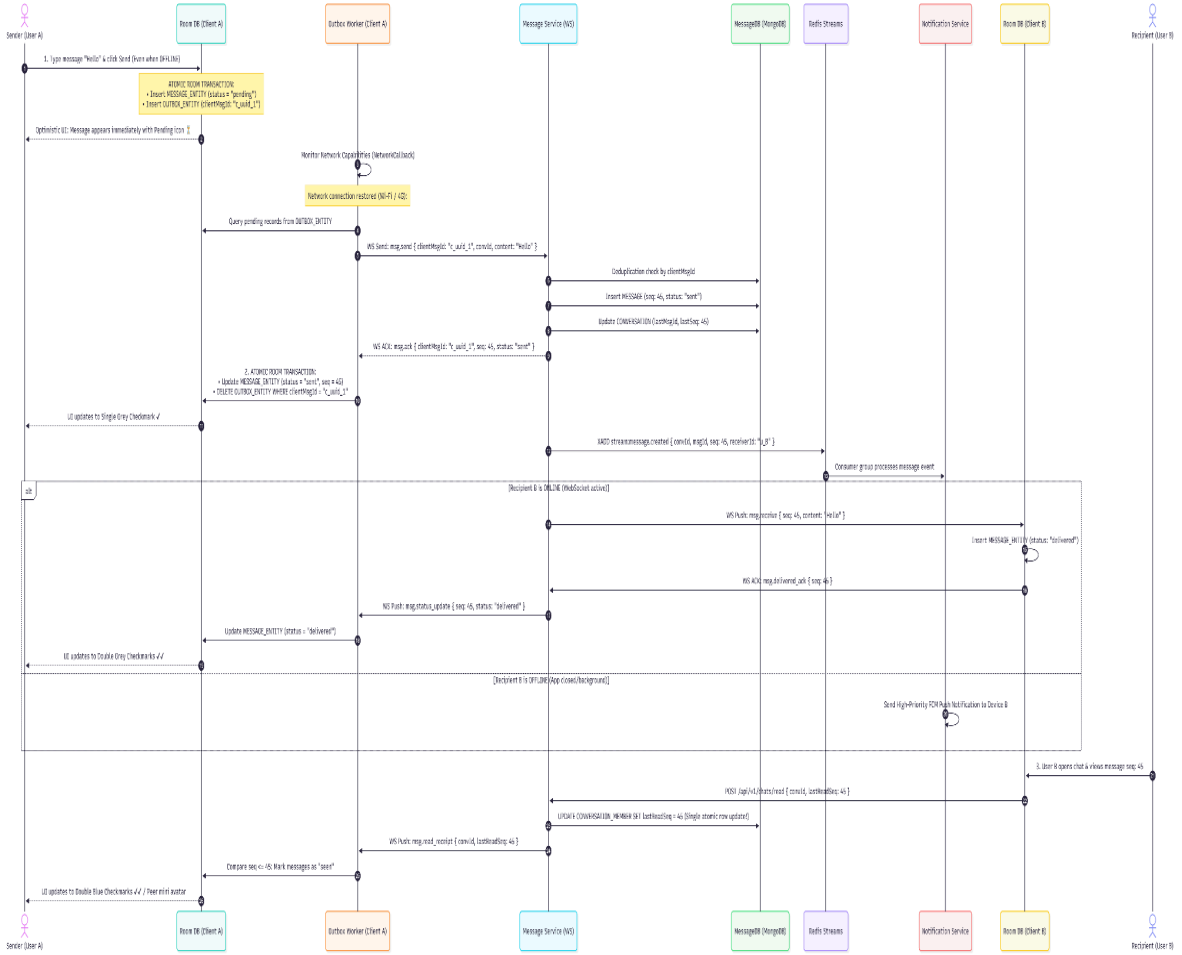
- Fill in team name, members, student IDs and supervisor under Document Information.
- Review every item marked (*) and confirm or remove it.
- Make sure every Must-priority FR has at least one corresponding AC.
- Align NFR figures (latency, connection count) with measured results during testing.
- Update the version and revision history after each change.



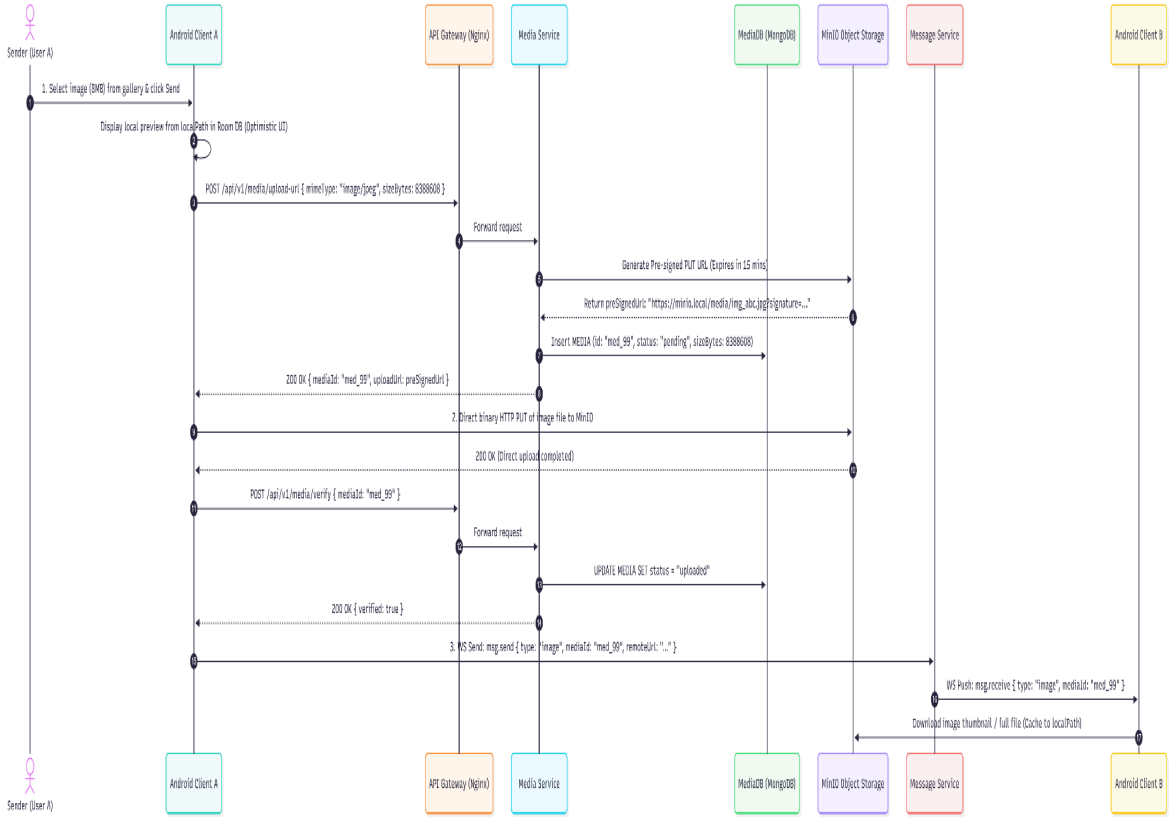
Seq1: Authentication & Device Registration Flow (OTP & JWT)



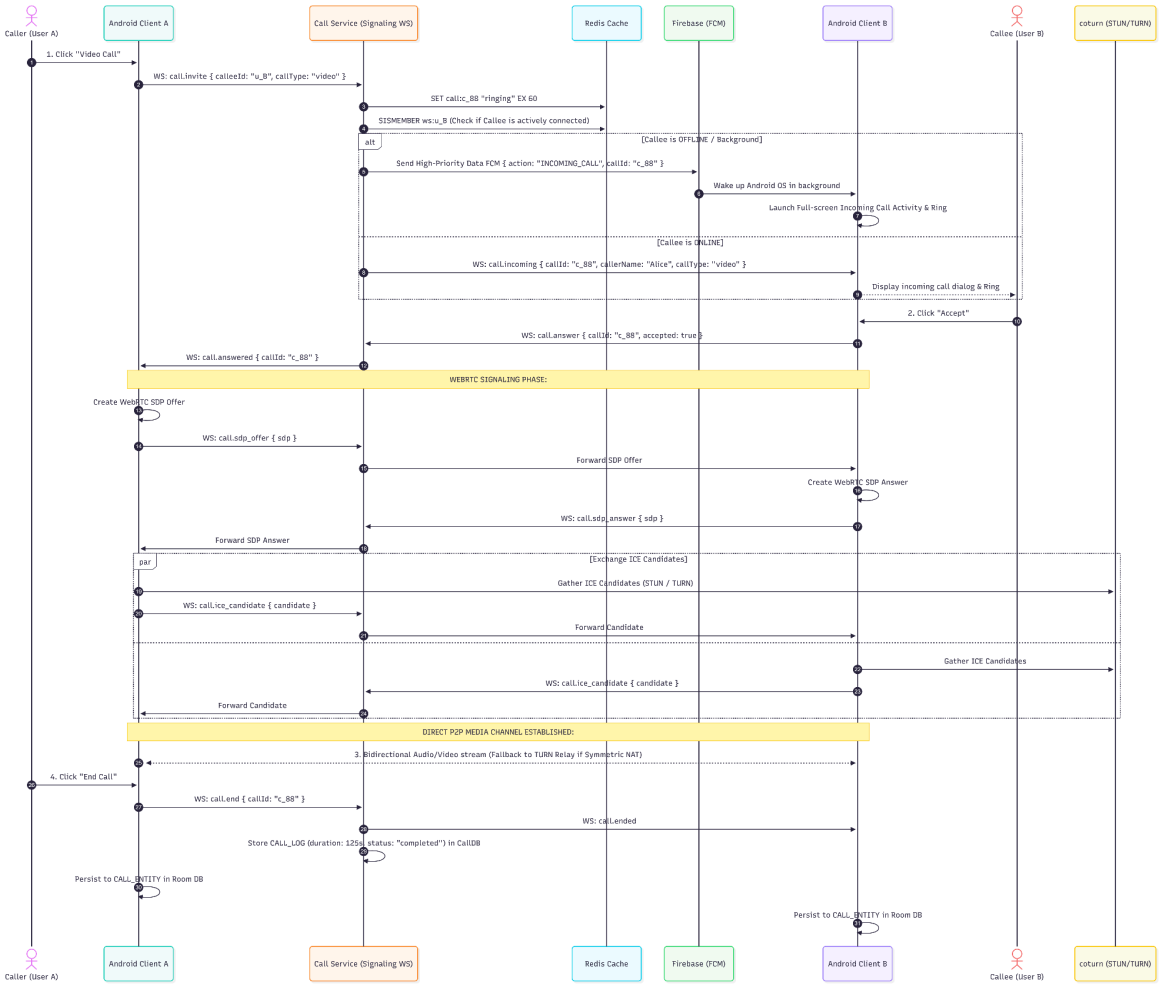
Seq2: Contact & Friendship Management Flow



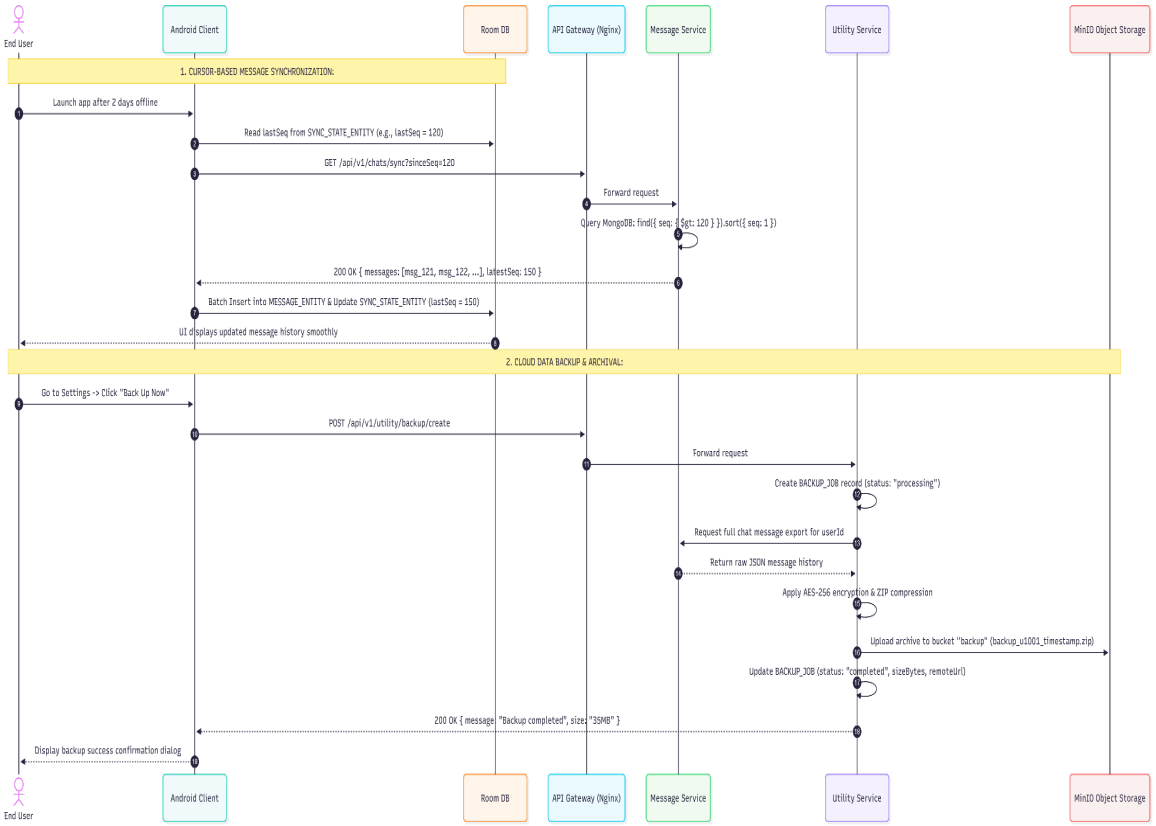
Seq3: Core 1-1 Messaging Flow with Offline Outbox & Read Receipts



Seq4: Media Upload via MinIO Pre-signed URL Flow



Seq5: WebRTC Audio/Video Call Signaling Flow



Seq6: Cursor-based Message Sync & Cloud Backup Flow

1. Traceability & Mapping Matrix

Diagram #	Workflow / Scope	Mapped Use Cases & Sequence Diagrams	Relevant DB Entities & Architectural Constraints
Diagram 1	**User Authentication & Device Registration (OTP & JWT)**	UC01, UC02, UC04, UC07 / **Seq1**	Redis `otp:{phone}` (TTL 5m), `otp_try:{phone}` (TTL 15m); `Identity.User`, `Identity.Device`; `Auth.Refresh_Token`.
Diagram 2	**Contact Discovery & Friendship Management**	UC08, UC09, UC10 / **Seq2**	`Identity.Contact` (`status: pending/accepted/blocked`); Redis Streams event `contact.accepted` triggers `CONVERSATION` creation.
Diagram 3	**Core 1-1 Messaging & Outbox Dispatcher**	UC11, UC13, UC14 / **Seq3**	**Implementation Note 1**: Android Room `OUTBOX_ENTITY`, NNN retry threshold, device network connectivity check, monotonic sequence numbering (`MESSAGE.seq`).
Diagram 4	**Read Receipts & Client-Side "Seen" Status Deduction**	UC14 / **Seq3**	**Implementation Note 2**: Server **does not** store a per-message "seen" status; it only records `CONVERSATION_MEMBER.lastReadSeq`. The client UI **deduces** the "Seen" state when `msg.seq <= peerLastReadSeq`.
Diagram 5	**Media Upload via MinIO Pre-signed URLs**	UC12 / **Seq4**	`Media.MEDIA` (`status: uploading -> ready`), MinIO Object Storage, client direct binary upload.
Diagram 6	**Audio/Video Call Signaling (WebRTC Signaling)**	UC20, UC21, UC22, UC22b / **Seq5**	Redis `call:{callId}` (60s ringing TTL), FCM High-Priority push, SDP/ICE candidate relay, coturn STUN/TURN fallback, `Call.CALL_LOG`.

Diagram 7	**Cursor-based Message Synchronization & Cloud Backup / Restore**	UC16, UC24, UC25 / **Seq6**	Room `SYNC_STATE_ENTITY`, `Utility.BACKUP_JOB`, MinIO encrypted archive, idempotent batch import using `clientMsgId`.

Diagram 1: User Authentication & Device Registration Flow (OTP & JWT)

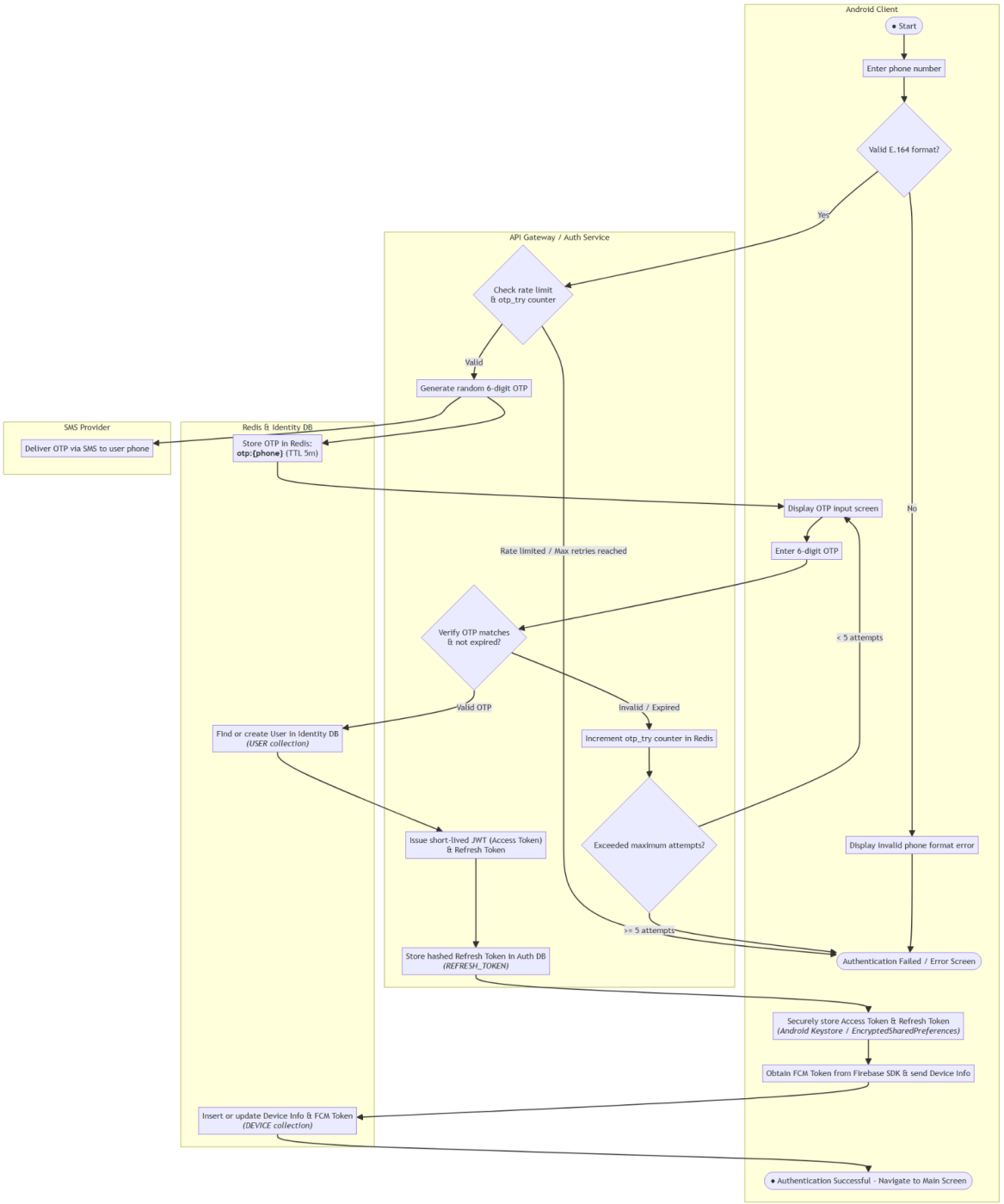

Android Client
• Start
Enter phone number
Valid E.164 format?
Yes
No
API Gateway / Auth Service
Check rate limit & otp_try counter
Valid
Rate limited / Max retries reached
Generate random 6-digit OTP
SMS Provider
Deliver OTP via SMS to user phone
Redis & Identity DB
Store OTP in Redis: otp:{phone} (TTL 5m)
Display OTP input screen
Enter 6-digit OTP
Verify OTP matches & not expired?
< 5 attempts
Valid OTP
Invalid / Expired
Find or create User in Identity DB (USER collection)
Increment otp_try counter in Redis
Issue short-lived JWT (Access Token) & Refresh Token
Exceeded maximum attempts?
Display invalid phone format error
>= 5 attempts
Store hashed Refresh Token in Auth DB (REFRESH_TOKEN)
Authentication Failed / Error Screen
Securely store Access Token & Refresh Token (Android Keystore / EncryptedSharedPreferences)
Obtain FCM Token from Firebase SDK & send Device Info
Insert or update Device Info & FCM Token (DEVICE collection)
• Authentication Successful - Navigate to Main Screen

Diagram 2: Contact Discovery & Friendship Management Flow

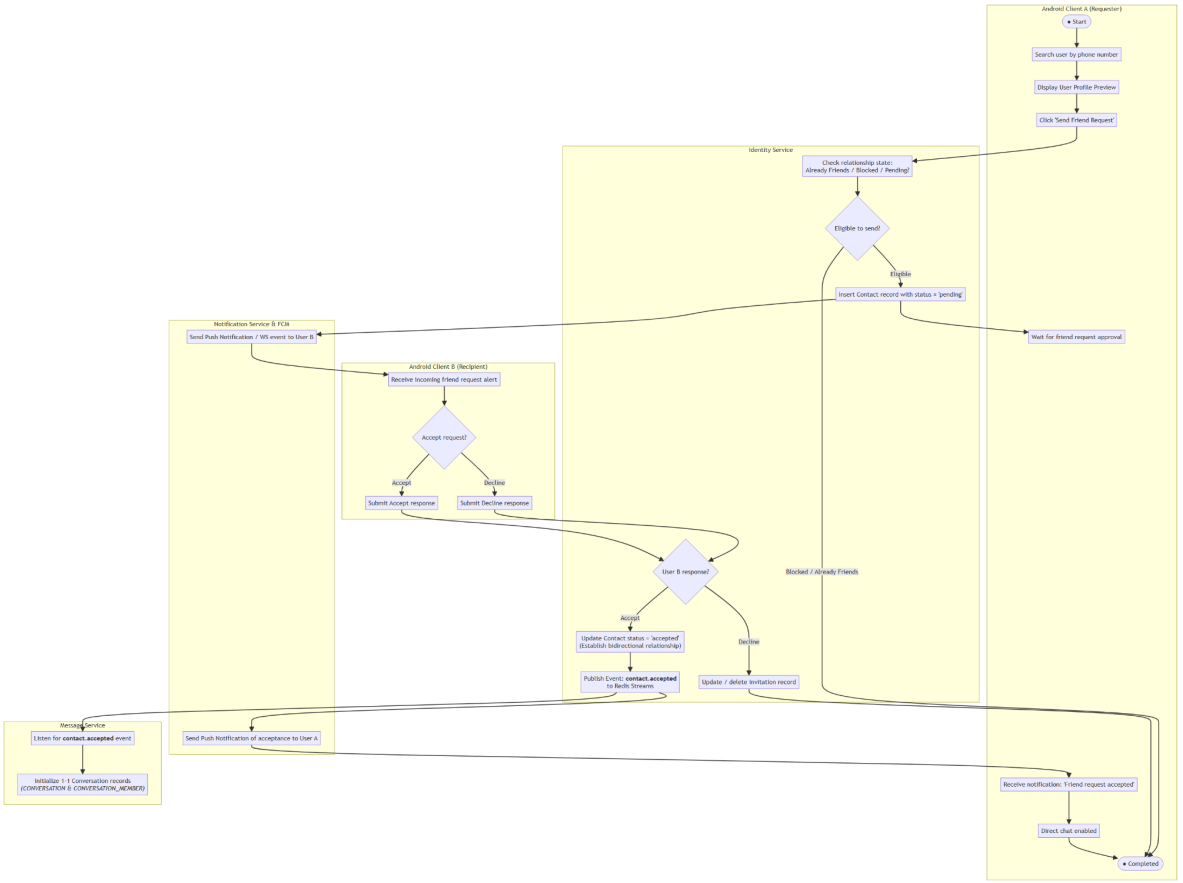


Diagram 3: Core 1-1 Messaging & Room Outbox Dispatch Workflow

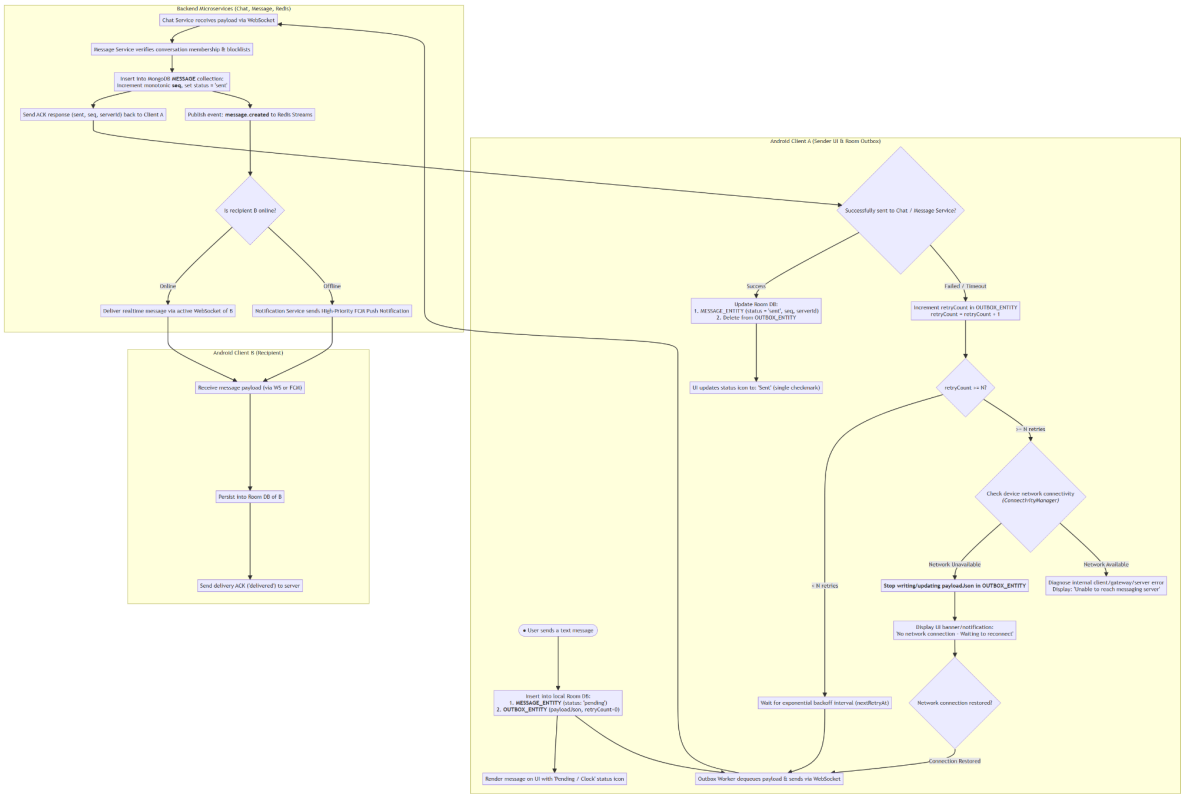


Diagram 4: Message Read Receipts & Client-Side "Seen" Status Deduction

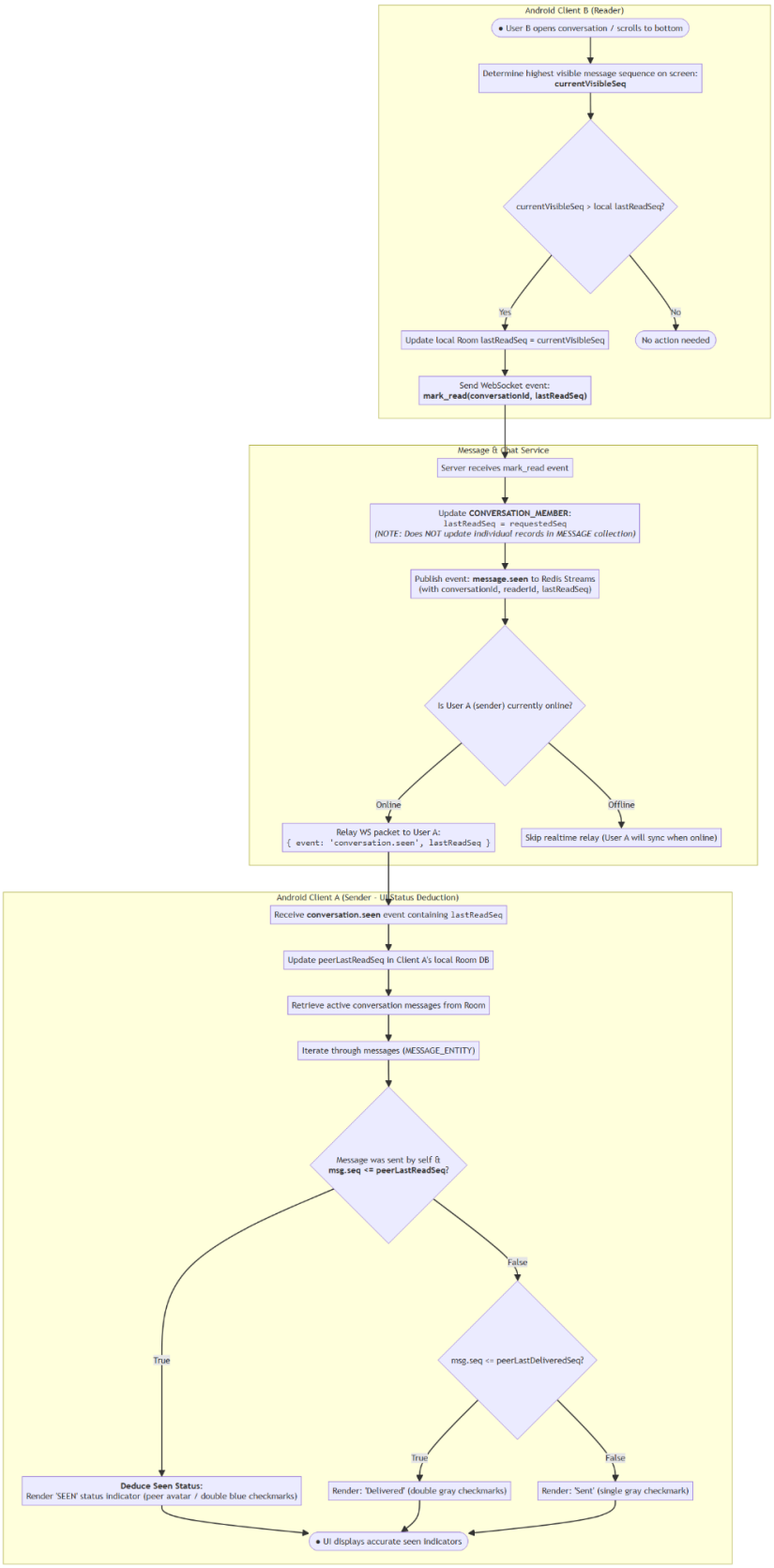


Diagram 5: Media Upload via MinIO Pre-signed URLs Flow

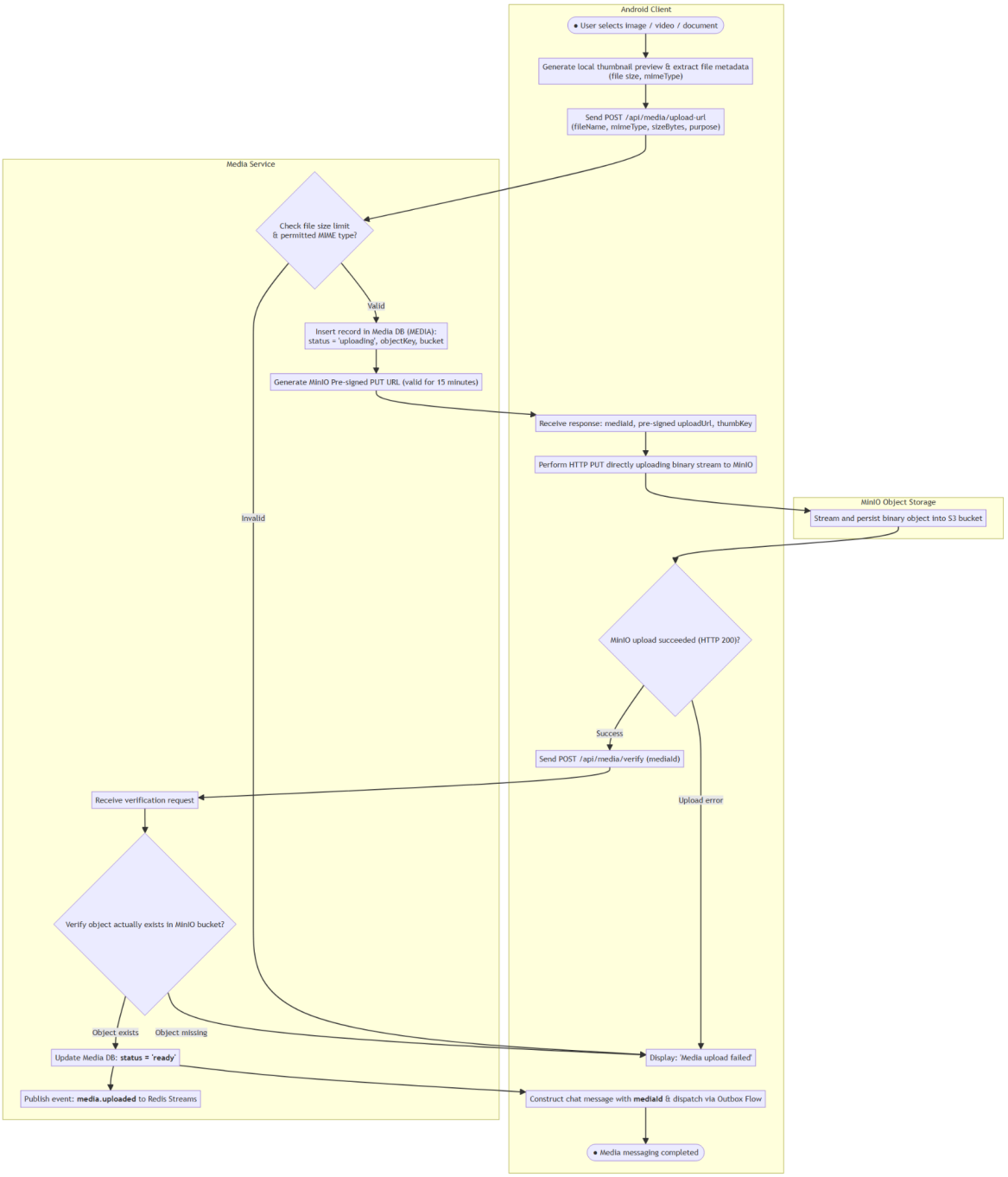


Diagram 6: WebRTC Audio/Video Call Signaling Flow

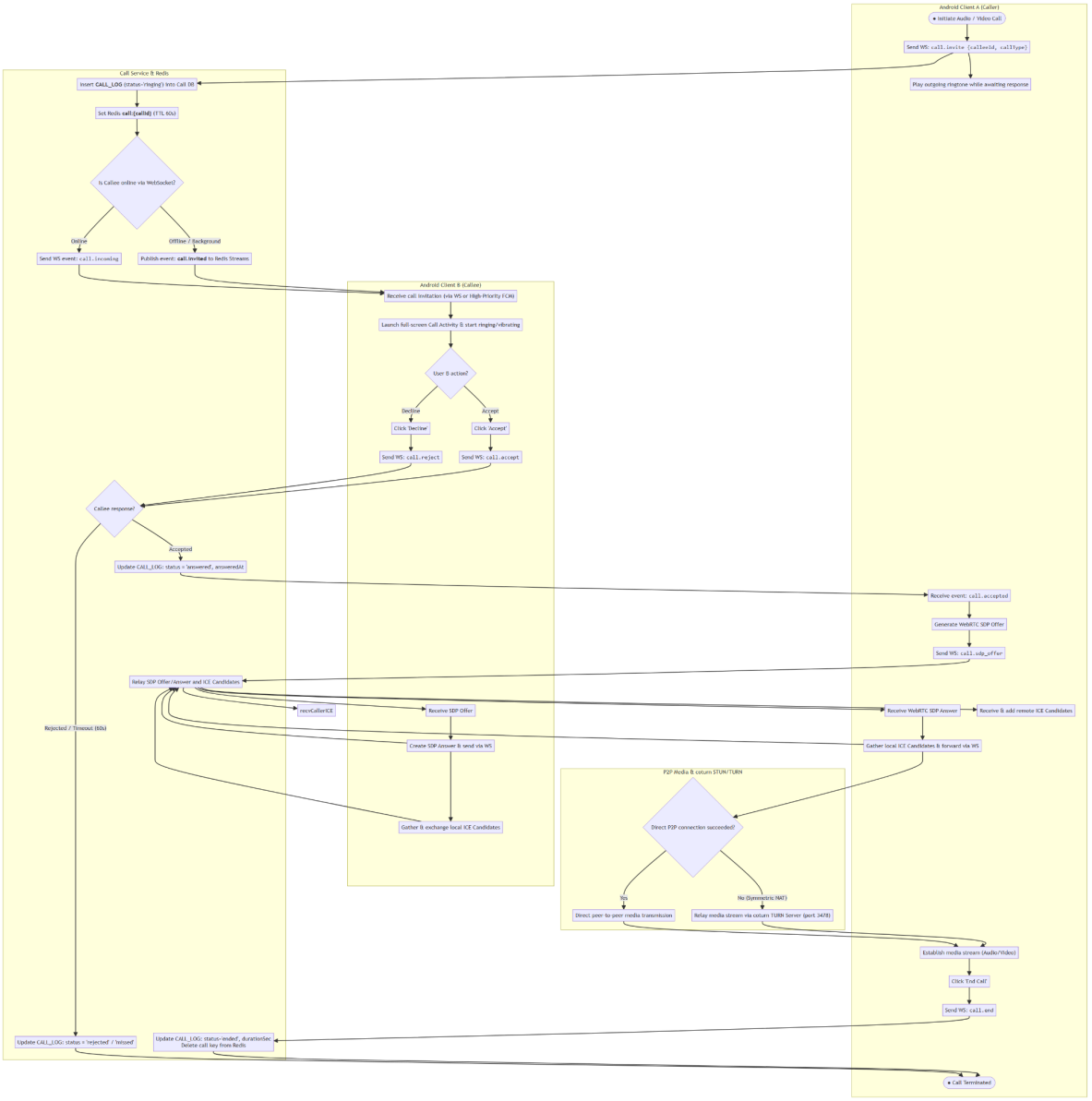
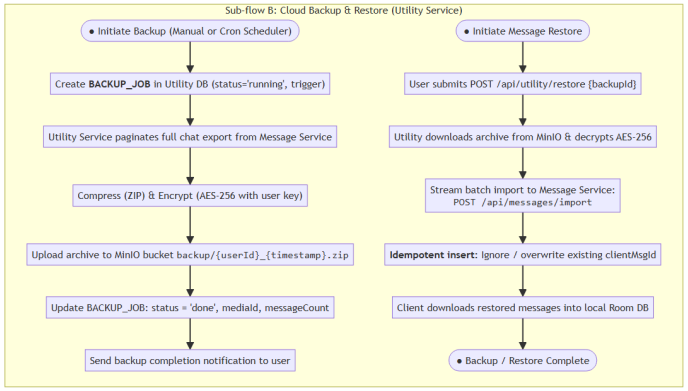
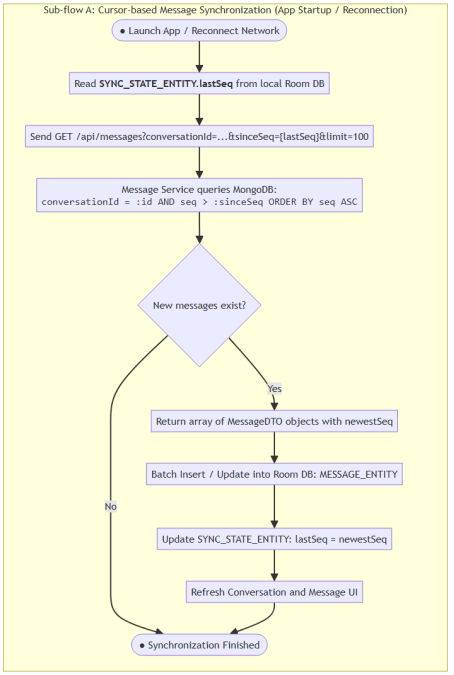
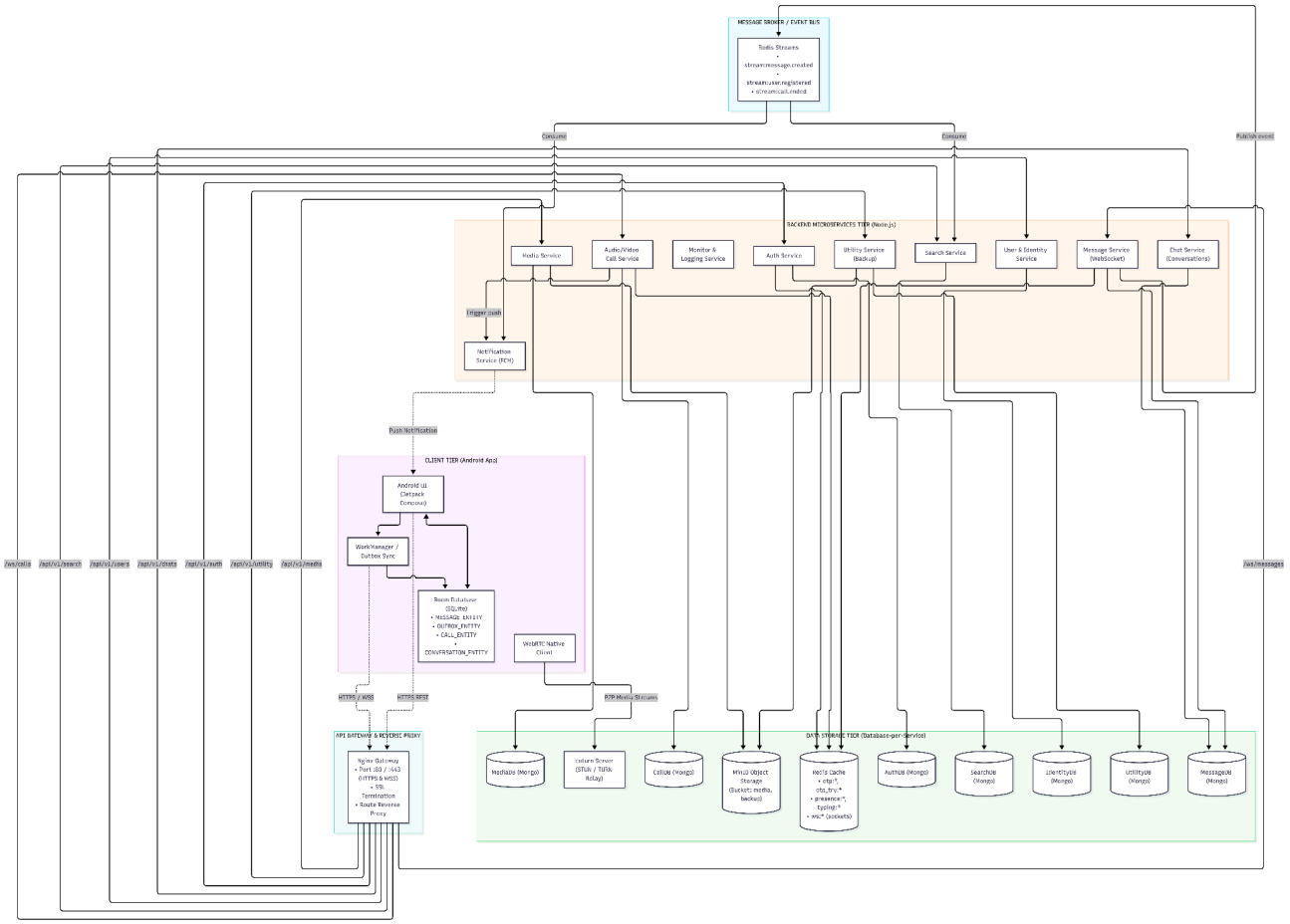


Diagram 7: Cursor-based Message Sync & Cloud Backup / Restore Workflow






MESSAGE BROKER / EVENT BUS
Redis Streams
• stream:message.created
• stream:user.registered
• stream:call.ended
Consume
Consume
Publish event
BACKEND MICROSERVICES TIER (Node.js)
Media Service
Audio/Video Call Service
Monitor & Logging Service
Auth Service
Utility Service (Backup)
Search Service
User & Identity Service
Message Service (WebSocket)
Chat Service (Conversations)
Trigger push
Notification Service (FCM)
Push Notification
CLIENT TIER (Android Java)
Android UI (Jetpack Compose)
WorkManager / Outbox Sync
Room Database (SQLite)
• MESSAGE_ENTITY
• OUTBOX_ENTITY
• CALL_ENTITY
• CONVERSATION_ENTITY
VoIP/ITC Native Client
/ws/calls
/api/v1/search
/api/v1/users
/api/v1/chats
/api/v1/auth
/api/v1/utility
/api/v1/media
/ws/messages
HTTPS / WSS
HTTPS REST
P2P Media Streams
API GATEWAY & REVERSE PROXY
Nginx Gateway
• Port 80 / 443 (HTTPS & WSS)
• SSL Termination
• Route Reverse Proxy
DATA STORAGE TIER (Database-per-Service)
MediaDB (Mongo)
coturn Server (STUN / TURN Relay)
CallDB (Mongo)
MinIO Object Storage (Buckets: media, backup)
Redis Cache
• otp:*
• otp_try:*
• presence:*
• typing:*
• ws:* (sockets)
AuthDB (Mongo)
SearchDB (Mongo)
IdentityDB (Mongo)
UtilityDB (Mongo)
MessageDB (Mongo)